

四川大學

本科生毕业论文(设计)



题 目	基于 eBPF 的开源 AI 推理引擎的跨语言调用链追踪方法
学 院	网络空间安全学院
专 业	网络空间安全
学生姓名	周杰
学 号	
指导教师	高凯

教务处制表
2026 年 6 月

基于 eBPF 的开源 AI 推理引擎的跨语言调用链追踪方法

专业：网络空间安全

学生：周杰

指导教师：高凯

摘要

摘要： 随着开源大语言模型在企业私有化部署中的广泛普及，以 Ollama 为代表的推理引擎采用 Go-CGO-C++ 三层混合编程与双进程解耦架构，在系统层面引入了前所未有的观测盲区：当恶意攻击者通过 API 发起目录遍历攻击或畸形载荷注入时，安全运维人员既无法追溯外部数据在推理引擎内部的完整流转路径，也无法回答该行为在内核层面触发了哪些系统调用。

本文设计并实现了 HybriChain——一个面向多语言推理引擎的跨语言调用链追踪与安全分析系统。针对跨语言符号映射难题，提出了五层符号分类体系 (f_0 - f_4)，将来自不同命名空间 (Go 运行时、CGO ABI、底层计算库) 的函数符号统一映射到同一语义空间；针对追踪开销难题，设计了基于 eBPF 的探针下沉策略，仅对推理计算核心函数挂载 uprobe，将运行时开销降至最低；针对噪声数据难题，设计了五步漏斗式降噪流水线，将原始事件流压缩 2-3 个数量级；针对调用链缝合难题，提出了基于时间戳对齐的置信度缝合算法，为每条跨语言边赋予 Confirmed/Inferred/Unknown 状态；在此基础上，构建了异常行为分析引擎，支持对 futex 阻塞、异常 mmap 分配、敏感路径访问等高危行为的自动化检测。

本文以 Ollama 为典型靶点验证了 HybriChain 的有效性。实验结果表明，该系统能够从原始内核事件流中还原出完整的跨语言调用链，准确识别推理引擎中的异常行为，为开源 AI 推理软件的可观测性与安全审计提供了新的技术手段。

关键词： eBPF；开源 AI 推理框架；跨语言程序分析；调用链追踪；Ollama；HybriChain

eBPF-Based Cross-Language Call Chain Tracing for Open-Source AI Inference Engines

Cyberspace Security

Student: Zhou Jie

Adviser: Gao Kai

Abstract

Abstract: Open-source large language models are now widely deployed in enterprise private environments. Inference engines like Ollama use a Go-CGO-C++ tri-layer hybrid architecture with dual-process decoupling. This design creates significant observability blind spots at the system level. When attackers exploit API endpoints for directory traversal or inject malformed payloads, security operators have no way to trace how external data flows through the inference engine internally, nor can they identify which kernel-level system calls the behavior triggers.

This thesis presents HybriChain—a cross-language call chain tracing and security analysis system built for multi-language inference engines. The cross-language symbol mapping problem is addressed through a five-layer symbol classification system (f_0 – f_4), which unifies function symbols from different namespaces—Go runtime, CGO ABI, and low-level computing libraries—into a shared semantic space. Tracing overhead is minimized by an eBPF-based probe sinking strategy that attaches uprobes only to core inference functions. Noisy data is handled by a five-step funnel noise reduction pipeline compressing raw event streams by 2–3 orders of magnitude. For call chain stitching, a timestamp-aligned confidence algorithm assigns Confirmed, Inferred, or Unknown states to each cross-language edge. On top of these components, an anomaly behavior analysis engine detects high-risk behaviors such as mutex blocking, abnormal mmap allocation, and sensitive path access.

Experiments using Ollama as the target validate the system. HybriChain reconstructs complete cross-language call chains from raw kernel event streams and identifies anomalous behaviors in inference engines. The results suggest a practical approach for observability and security auditing of open-source AI inference software.

Key Words: eBPF; open-source AI inference framework; cross-language program analysis; call chain tracing; Ollama; HybriChain

目录

摘要	I
Abstract	I
第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.3 论文主要工作	2
1.4 论文组织与结构	3
第 2 章 相关技术综述	4
2.1 大语言模型推理架构的硬件极限与系统级突围	4
2.1.1 底层硬件制约与架构分析	4
2.1.2 应对内存危机与边缘部署方案	4
2.1.3 AI 基础设施的安全脆弱性暴露	5
2.2 eBPF: 重塑 Linux 内核与微服务可观测性范式	5
2.2.1 eBPF 基础架构与核心数据传输	5
2.2.2 将 eBPF 推向应用层服务质量与 AI 黑盒	6
2.2.3 内核态观测所带来的系统级安全对抗	6
2.3 跨语言系统边界的消解与全栈漏洞挖掘分析	7
2.3.1 Go-CGO 互操作与混合图谱重建策略	7
2.3.2 追踪多语种静态数据流（污点分析）的进阶演化	7
2.4 动态二进制插桩体系的底层安全基石作用	8
2.4.1 DBI 的轻量级透明性与架构统一	8
2.4.2 内存级别极端监控的”影子”机制确立	8
2.5 本章小结	8
第 3 章 HybriChain 系统设计	10
3.1 问题定义与研究目标	10
3.1.1 核心问题	10
3.1.2 系统目标	10
3.2 系统总体架构	11
3.3 高效且完备的函数调用追踪	12
3.4 高效统一的调用链合成与降噪	13
3.5 准确的跨语言符号映射	14
3.6 异常检测规则引擎设计	15
3.7 本章小结	16
第 4 章 HybriChain 实现方案	17
4.1 Ollama 推理引擎的目标特征	17
4.2 高效且完备的函数调用追踪的实现	17

4.3 高效统一的调用链合成与降噪的实现	19
4.4 准确的跨语言符号映射的实现.....	21
4.5 异常检测规则的具体实现.....	23
4.6 本章小结.....	24
第 5 章 实验设计与结果分析.....	26
5.1 实验环境.....	26
5.2 调用链缝合准确性验证.....	26
5.2.1 Ground Truth 基准数据集.....	26
5.2.2 缝合评估结果	27
5.2.3 缝合结果 Listing.....	27
5.3 eBPF 追踪对推理性能的影响评估.....	27
5.3.1 推理时延 (TTFT) 影响分析	27
5.3.2 内存占用分析	28
5.4 数据清洗流水线效率分析.....	29
5.4.1 事件压缩效果	29
5.4.2 分析处理速度	29
5.4.3 过滤正确性验证	30
5.5 系统调用分布与推理阶段分析.....	31
5.5.1 系统调用分布特征	31
5.5.2 推理阶段划分	31
5.6 异常行为发现与案例分析.....	32
5.6.1 异常告警检测结果	32
5.6.2 典型案例分析	33
5.7 HybriChain 综合评估.....	33
5.8 本章小结.....	34
第 6 章 总结与展望.....	35
6.1 论文工作总结.....	35
6.2 研究局限与不足.....	35
6.3 未来工作展望.....	36
6.4 本章小结.....	36
参考文献	37
声 明.....	39
学位论文使用授权书	39
四川大学本科毕业论文 AI 工具使用声明.....	40
致 谢.....	41

第 1 章 绪论

1.1 研究背景与意义

近年来，以大语言模型（Large Language Model, LLM）为代表的开源人工智能软件迎来了爆发式的增长，彻底重塑了计算基础设施的演进方向。为了满足数据主权、商业机密保护与隐私合规的苛刻要求，在边缘计算设备和企业内部私有化部署大语言模型逐渐成为主流。在此背景下，开源社区涌现出了如基于纯 C/C++ 实现的 llama.cpp^[1] 及其底层张量代数库 ggml^[2] 等极其优秀的轻量级推理引擎。

然而，大语言模型的参数规模呈现出指数级的膨胀，将底层硬件的内存带宽与计算能力的博弈推向了极端的白热化阶段。X. Ma 与计算机体系结构泰斗 D. Patterson 在其合作的综述中，明确将这种由大规模权重加载与解码（Decode）阶段自回归特性所引发的“内存受限（Memory Bound）”定义为现代 AI 硬件面临的严峻“推理危机”^[3]。A. M. Jain 等人的实证研究进一步表明，在生产级聊天机器人架构中，随着并发请求的扩展，系统尾部延迟急剧上升，硬件算力的物理极限已被触及^[4]。为了应对这种危机，底层系统软件层面开始主导架构重构。W. Kwon 等人提出了教科书级别的 PagedAttention 机制，将现代操作系统的虚拟内存与分页管理思想引入 AI 领域，彻底改变了键值缓存（KV Cache）的内存分配逻辑^[5]。这种由软硬件协同带来的极致性能优化，不可避免地导致了现代 AI 推理框架在系统架构上极度复杂，普遍采用多进程解耦与多语言混合编程范式（如 Go、Python 与 C/C++ 混编）。

这种极其复杂的软件工程实践，使得推理引擎内部的数据流转行为对网络空间安全运维人员而言几乎完全不透明。随着这些多语言框架在生产环境中的广泛应用，其针对数据传输应用编程接口（API）的安全脆弱性逐渐暴露。例如，在 PyTorch 官方支持的 TorchServe 模型部署工具中，由于其架构允许未经严格身份验证的远程用户通过 API 上传模型存档，导致了在反序列化阶段触发极其严重的远程代码执行（RCE）漏洞（CVE-2023-43654）^[6]。类似地，安全研究机构 Wiz Research 披露了 Ollama 框架的 CVE-2024-37032（Problama）高危漏洞^[7]。该漏洞的底层成因是，Ollama 守护进程在处理高层 API 传输的模型数据摘要（Digest）时，未能严格验证其十六进制格式规范，攻击者通过构造携带路径穿越符的畸形 API 数据，成功绕过系统沙箱，实现了对宿主机底层任意核心文件的越权读取。

上述真实的工业界安全事件暴露了一个根本性的技术断层：在多语言混合编程架构下，从顶层应用 API 接收数据负载，到中层编程语言（如 Go 到 C++）跨界桥接序列化，再到底层操作系统内核调用的完整链路，在现有可观测工具下无法被连贯还原。当发生恶意入侵时，系统无法回答“外部注入的数据最终触碰了哪些内核物理内存”。因此，开展针对开源 AI 软件的数据传输 API 调用链分析，实现从高层语义到低层内核执行的完整追踪与异常行为审计，对提升下一代开源大模型应用底座的安全性、稳定性与可观测性，具有极其迫切的理论价值与工程指导意义。

1.2 国内外研究现状

在应对庞杂系统的可观测性与安全审计问题上，学术界与工业界在动态内核追踪、静态跨语言程序分析以及动态二进制插桩等领域展开了大量探索，但针对多语言大模型架构的数据流分析依然存在显著的方法论缺失。

在内核动态追踪与系统可观测性领域，扩展的伯克利数据包过滤器（eBPF）经历了革命性的演进。自 A. Starovoitov 提交核心补丁将其重塑为通用内核虚拟机以来^[8]，B. Gregg^[9] 与 D. Calavera^[10] 奠定了其在系统性能剖析领域的主导地位。随着 A. Nakryiko 引入 BPF Ring Buffer 解决并发数据无损回传^[11]，以及 Cilium 团队开源纯 Go 语言的 eBPF 交互库^[12]，其应用边界不断拓宽。Y. Hu^[13]、M. Ibnath^[14] 以及 B. Zou^[15] 等研究者相继将 eBPF 应用于无服务器（Serverless）网络监控、微服务流式延迟测量以及大语言模型底层的算子级硬件剖析。在安全防护层面，ControlPlane^[16]、LinuxSecurity^[17] 与 Groundcover 团队^[18] 对 eBPF 的威胁模型及替代传统 Auditd 的微隔离实践进行了深度论述。

在跨语言边界的程序分析方面，由于语言环境的割裂，控制流极易断裂。针对 Go 语言调用 C/C++ 的互操作机制，Go 官方团队的 CGO 工具链^[19] 与 J. Chen 等人的实证研究^[20] 揭示了其在类型安全上的巨大隐患。M. Hu 等人^[21]、中科大安全实验室的 POLYCALL 项目^[22] 以及 R. Sunil^[23] 分别尝试通过图转换算法、WebAssembly 中间表示以及大语言模型智能推断来重建多语言混合图谱。同时，在跨语言数据流（污点分析）领域，Brucker^[24]、CHARON 团队^[25] 以及 X. Cheng 提出的 Deepwukong 图神经网络漏洞检测模型^[26]，展示了在静态环境下溯源跨界污染流的巨大潜力。

在作为系统防御终极基石的动态二进制插桩（DBI）体系中，D. Bruening 的 DynamoRIO^[27]、C. Lattner 的 LLVM 架构^[28] 以及 N. Nethercote 开发的 Valgrind^[29] 及其影子内存核心机制 Memcheck^[30]，确立了程序运行时极端监控的理论体系。然而，大模型推理场景在算力与内存吞吐上的极致要求，使得上述重量级的分析方法面临着严重的落地困境。

1.3 论文主要工作

本文聚焦于开源 AI 软件的数据传输 API 调用链分析这一前沿且复杂的系统级难题，针对混合编程架构带来的控制流断裂与观测盲区，设计并实现了一套动静结合、兼顾极低开销与高语义保真度的底层追踪与安全审计框架。论文的主要创新与研究工作包括：

第一，构建跨语言数据传输符号异构映射机制。针对 API 载荷在 Go 语言 CGO 编译期遭遇的符号哈希加扰与 C++ 端复杂的名字重整（Name Mangling）导致的控制流断崖，设计多级抽象符号映射算法。通过深度遍历二进制符号表并提取编译器链接元数据，成功建立从高层路由处理 API 到底层计算执行算子的等价映射链路，修复了多语言程序的静态拓扑。

第二，实施轻量级 eBPF 探针深度下沉战略。摒弃传统的进程级拦截思路，将 eBPF 探针精准挂载于大语言模型推理进程的数据流转咽喉节点。实现了对 API 跨进程通信（IPC）载荷序列化、底层大规模张量 mmap 内存映射调度以及并发线程 futex 锁机制的纳秒级极

低开销捕获。

第三，提出动静结合的 API 调用链融合缝合算法。以纳秒级高精度系统时间戳与线程上下文 (PID/TID) 为纽带，将提取的静态跨界符号图谱与动态 eBPF 环形缓冲区上报的事件轨迹进行高维状态融合。通过基于滑动时间窗口的匹配模型，自动补全丢失的跳转边，生成端到端具备强因果逻辑的数据传输 API 调用链。

第四，开发基于底层调用链的异常安全分析引擎。基于缝合后的高质量完整数据流转轨迹，提炼大模型推理场景下的核心威胁模式。设计多维度规则过滤矩阵，实现了对类似 Problama 的目录遍历越权访问、畸形载荷导致的级联内存锁死等高危异常 API 行为的自动化阻断与研判。

1.4 论文组织与结构

本文共分为六个章节，各章内容安排如下：

第 1 章为绪论，阐述大语言模型推理引擎的硬件极限挑战与多语言混合编程带来的安全可观测性断层，明确本文的研究背景与意义，并对国内外相关研究现状进行综述。

第 2 章为相关技术综述，系统剖析主流推理引擎的多语言混合架构、eBPF 内核可观测性技术、跨语言程序静态分析方法以及动态二进制插桩体系，评述现有技术在应对多语言 AI 安全审计时的局限。

第 3 章为 HybriChain 系统设计，从问题定义出发，阐述“为了 xxx 所以做 xxx”的论证主线——为了获得跨语言调用链，首先实现高效且完备的函数调用追踪；为了让原始数据可用，进行高效统一的调用链合成与降噪；为了让缝合结果可解释，建立准确的跨语言符号映射；最后在上述成果基础上设计异常检测规则引擎。

第 4 章为 HybriChain 实现方案，以 Ollama 推理引擎为典型靶点，与第三章的设计方案形成一一对应的闭环，详细阐述各模块的具体实现与关键算法落地。

第 5 章为实验评估，在多并发高负载的大模型场景下，对 HybriChain 的调用链还原准确率、置信度分布、追踪性能开销以及异常检测能力进行定量验证。

第 6 章为总结与展望，对全文工作进行总结，梳理本文的主要贡献与局限性，并对下一代系统可观测性技术进行展望。

第 2 章 相关技术综述

为了实现对开源大语言模型复杂系统内部的深度穿透，准确实施针对数据传输 API 的调用链分析，本章将结合现代计算基础设施前沿技术的演进轨迹，对涵盖大模型推理硬件限制、eBPF 内核可观测性、跨语言程序图谱构建以及动态二进制底层追踪等领域的核心学术文献与工业界标杆进行穷尽式剖析。特别强调每一项关键研究在其技术节点上究竟解决了哪些核心挑战，以及在应对多语言 AI 安全审计时暴露出何种未决的局限性。

2.1 大语言模型推理架构的硬件极限与系统级突围

现代大模型的迭代极大地放大了计算架构中的根本性矛盾，迫使开发者在系统软件层进行极其复杂的工程优化，这些优化构成了跨语言调用链追踪的客观背景。

2.1.1 底层硬件制约与架构分析

X. Ma 与 D. Patterson^[3] 详细剖析了大型语言模型推理硬件的发展趋势与物理局限，他们通过对高带宽内存（HBM）堆叠成本与算力之间博弈的量化研究，解决了学术界对于“内存墙（Memory Wall）”制约大模型算力发展的宏观认知问题，明确指出了单纯堆砌硅片的路径不可持续。但其研究的局限在于，该分析高度聚焦于纯芯片与硬件互联拓扑的物理特性探讨，未能给出应用层系统软件突破这些瓶颈的具体代码级干预方案。

A. M. Jain 等人^[4] 通过构建合成负载生成器与统计学模型评估架构扩展性，解决了预测生产级聊天机器人在高并发长文本输入下发生尾部延迟（Tail Latency）爆炸的定量难题。但还有待解决的问题是，其研究将 AI 框架本身视为黑盒系统，仅在宏观负载层面测量输出，无法解释延迟激增在系统内部引发的跨层级拥塞机理。

2.1.2 应对内存危机与边缘部署方案

W. Kwon 等人^[5] 提出了著名的 PagedAttention 机制，他们利用操作系统经典的虚拟内存按需分页（On-demand Allocation）哲学，彻底解决了 LLM 推理时海量 KV Cache 必须连续分配导致的外部物理内存碎片化问题，实现了吞吐量的几何级攀升。然而不足之处在于，该机制极度优化了内存复用率，却缺乏对跨并发请求在混合内存块池中相互越界读写的深入安全隔离探讨。

G. Gerganov^[1,2] 开发了具有划时代意义的纯 C/C++ 推理库 llama.cpp 及其核心张量库 ggml。他们通过实现深度的整数级混合量化（Quantization）技术与全平台指令集优化，解决了在缺乏专用 GPU 的边缘设备与消费级处理器上部署千亿级参数大模型的难题。但该框架面临的工程挑战是，由于追求极致底层的纯 C/C++ 控制，导致其难以直接且安全地对接现代基于 Go 或 Python 的高并发网络微服务 API 层，从而衍生出了错综复杂的跨语言封装层。

图 2.1 展示了这一架构的典型形态：Ollama 在 Go HTTP Handler 层接收 API 请求后，经由 CGO 绑定层自动生成的桥接函数，最终调用底层 llama.cpp C++ 层完成推理计算。这种跨越 Go → CGO → C++ 三层语言的调用链，正是 HybriChain 需要追踪的核心对象。

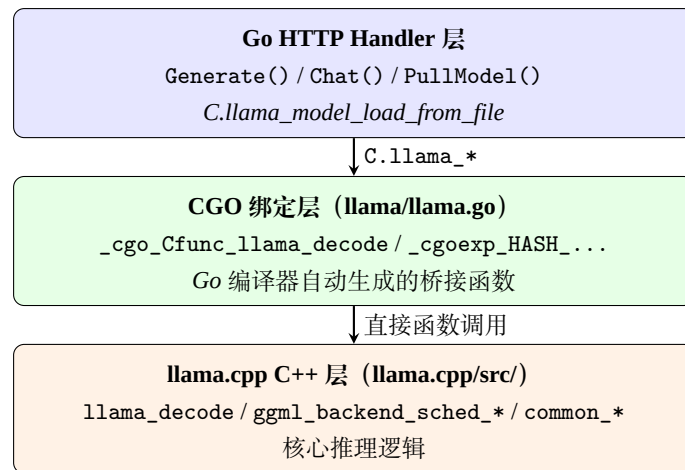


图 2.1 Ollama 混合编程架构示意

2.1.3 AI 基础设施的安全脆弱性暴露

PyTorch 团队维护的 TorchServe 项目^[6]作为标杆性的部署工具，解决了深度学习模型提供 REST/gRPC API 与多模型并发动态批处理的业务痛点。但其在演进中暴露出的问题是，由于追求接口的绝对灵活，早期架构在反序列化阶段允许动态模型注册，导致遭遇了类似 CVE-2023-43654 这样通过构造恶意 YAML 存档进行远程代码执行（RCE）的毁灭性漏洞。

S. Tzadik 领导的 Wiz 安全团队^[7]详细披露了 Ollama 框架的 CVE-2024-37032 高危漏洞（Problama）。他们分析了攻击者如何利用该系统底层的解析缺陷，解决了业界对该框架在获取模型 API 数据包摘要（Digest）时由于未严格验证十六进制格式规范而引发“目录遍历攻击”的具体机理认知。但现阶段亟需解决的系统性难题是，传统安全修复仅停留在事后的打补丁与特定路径过滤上，完全未能在底层内核层面建立一套自动化、实时溯源此类畸形 API 数据从高层网络插口流动到敏感系统文件读写的监控追踪链路。

2.2 eBPF：重塑 Linux 内核与微服务可观测性范式

为了从根本上穿透大模型框架的双进程与跨语言壁垒，eBPF 技术展现出了独一无二的优越性，其演进史本身就是一部寻求在系统性能与监控深度之间完美平衡的进化史。

2.2.1 eBPF 基础架构与核心数据传输

A. Starovoitov^[8]提交的一系列历史性重构补丁，解决了传统 BPF 只能受限于网络子系统的僵化局面，通过将其升格为通用内核虚拟机并引入极其严格的代码静态验证器（Verifier），赋予了用户态在内核极速且安全执行观测逻辑的能力。基于此架构，B. Gregg^[9]和 D. Calavera^[10]分别解决了传统系统性能剖析与网络层监控无法兼顾纳秒级开销与全栈透视的工程痛点，奠定了可观测性新范式。

为了突破高频追踪数据的回传瓶颈，A. Nakryiko^[11]引入了划时代的 BPF Ring Buffer 数据结构。他通过构建多生产者单消费者（MPSC）共享内存队列，彻底解决了早期 Perf Buffer 在多核高并发大流量下频繁发生的事件拥塞丢失与时间戳乱序（Event Re-ordering）

乱象。随后, Cilium 与 Cloudflare 联合团队^[12]开发了 cilium/ebpf, 解决了云原生环境中 Go 语言无法高效、原生地下发与解析 eBPF 字节码的障碍。但这些底层能力依然存在局限, 即纯粹的数据传输通道自身并不具备对复杂 AI API 负载进行跨语言重组的高级语义感知能力。

表 2.1 从执行位置、性能开销、安全验证、Go 兼容性等维度, 将 eBPF 与 strace、GDB、DBI (Pin)、perf 等传统追踪技术进行了横向对比。如表中所示, eBPF 在内核态安全沙箱中运行, 由 Verifier 提供运行时安全验证, 同时具备极低的性能开销与对 Go 1.17+ 良好的兼容性, 这些特性使其成为追踪多语言混合推理引擎的最佳候选技术。

表 2.1 eBPF 与其他追踪技术对比

维度	strace	GDB	DBI (Pin)	eBPF	perf
执行位置	用户态 (ptrace)	用户态	用户态 (运行时注入)	内核态 (安全沙箱)	内核态
性能开销	高	极高	高	低	低
安全验证	无	无	无	有 (verifier)	有
Go 1.17+ 兼容	可用	需调试信息	危险 (ABI 不兼容)	推荐 (无侵入)	可用
学习曲线	低	高	高	中	中

2.2.2 将 eBPF 推向应用层服务质量与 AI 黑盒

Y. Hu^[13]创新性地 将 eBPF uprobe 探针应用于 WebAssembly (Wasm) 运行时, 解决了在无服务器 (Serverless) 沙箱环境中无法侵入源码即可进行精准 HTTP 流量统计的盲区问题。M. Ibnath 等人^[14]提出的 eBeeMetrics 框架, 解决了系统在缺乏离线反馈分析的条件下, 跨多种网络协议精准评估微服务吞吐与尾部延迟的难题。但以上宏观观测研究均存在缺陷, 即将庞大的进程视为无状态的黑盒中继站, 无法深入探究数据变量在跨语言进程内部的流向。

针对大型人工智能模型底层的黑盒痛点, B. Zou 等人^[15]提出的 ProfInfer 框架实现了史无前例的降维打击。他们利用 uprobe 结合硬件性能监控计数器 (PMC), 解决了在 Token 生成解码粒度上, 精确追踪底层张量算子 (如矩阵乘法) 导致 L3 缓存未命中 (Cache Refill) 等微观硬件物理指标的难题。然而令人遗憾的是, 该系统的高度专一化导致其完全放弃了对顶层数据传输 HTTP API 与底层 C++ 计算之间跨语言反序列化语义的缝合, 当面对应用层 API 发起的利用攻击时, 该框架无法提供任何溯源保护。

2.2.3 内核态观测所带来的系统级安全对抗

ControlPlane 的系统白皮书^[16]与 LinuxSecurity 的深度对比分析^[17], 解决了业界在选择传统日志记录器 (如 Auditd) 与 eBPF 微隔离方案时的理论分歧。他们指出 Auditd 的事后诸葛亮机制无法抵御现代级联攻击, 而 eBPF 在指令执行前 (Pre-execution) 拥有微秒级阻断优势。同时, Groundcover 团队^[18]解决了 eBPF 在复杂 Kubernetes 容器网格环境中的高可用部署最佳实践问题。但安全社区依然面临挑战: 由于 eBPF 的绝对权力, 如何防范恶意构造的 BPF 程序进行拒绝服务攻击或越权读取, 必须引入极其克制的最小特权 (Least

Privilege) 模型, 而当前针对 AI 系统内部特定 API 行为定制的轻量级细粒度隔离策略尚属空白。

2.3 跨语言系统边界的消解与全栈漏洞挖掘分析

”多语种编程”是当代复杂架构的常态, 但不匹配的类型系统与异构编译器, 直接导致了传统单语言静态分析的集体失效, 学术界对此展开了猛烈的技术攻坚。

2.3.1 Go-CGO 互操作与混合图谱重建策略

Go 官方团队长期维护的 CGO 核心工具链^[19], 解决了 Go 这门具有自动垃圾回收 (GC) 特性的云原生语言与系统级 C 代码进行底层交互的互操作基础问题。针对该机制的工程滥用, J. Chen 等人^[20]开展了海量的开源项目实证扫描。他们总结出多达 19 种特定于 CGO 的致命缺陷模式, 彻底解决了学术界对 Go 语言中滥用互操作会导致类型安全遭到破坏、指针逃逸以及引发严重内存崩溃的感性认知, 提供了坚实的量化数据。但其研究局限于静态规则扫描阶段, 一旦程序编译打包, 静态工具便无法在运行时的复杂并发环境中拦截真实的越界访问。

为了跨越控制流断层, M. Hu 等人^[21]提出了一种极具创意的图转换 (Graph Transformation) 算法。他们通过提取不同语言接口层的抽象模型并执行节点融合, 解决了在诸如 Python 调用动态 C 库 (.so) 时的外部映射断链问题。但该方法的严重局限在于, 其依赖于动态链接库中暴露的标准导出符号表; 由于大语言模型推理框架追求极致效率, 多采用静态链接 (Static Linking) 将 Go 与 C++ 打包, 导致该方法完全失效。

在此背景下, 中科大安全实验室提出的 POLYCALL 框架^[22]采取了降维打击。他们主张直接将所有高级语言统一下沉编译至 WebAssembly (Wasm) 中间表示, 成功解决了因为语法差异巨大而无法进行统一函数索引的难题, 挖掘出了数百倍于源码分析的隐藏调用边。但其未能解决的挑战是, 静态图谱极其庞大且缺乏时间维度, 无法捕获实际高频调用中每个 API 载荷的真实状态分布, 且极难处理现代编译器故意强行注入的随机哈希加扰 (Hash Mangling)。此外, R. Sunil^[23]尝试引入大型语言模型本身的强大内化知识库, 解决了人工推导跨语言映射规则极其低效的问题。但目前的致命缺陷在于, 大语言模型生成的控制流图经常出现严重的”推理幻觉”, 其产出的拓扑置信度远未达到网络安全领域所需的苛刻取证标准。

2.3.2 追踪多语种静态数据流 (污点分析) 的进阶演化

针对跨域数据流动引发的沙箱逃逸, 早期 Brucker 团队的研究^[24]解决了揭示混合移动架构 (Hybrid Mobile Apps) 下, 网页端 JavaScript 恶意向底层原生组件传输异常数据流带来风险的问题。

随后, CHARON 框架^[25]标志着此类技术真正走向成熟。他们开创性地在内存中动态链接不同语言的代码属性图 (CPG), 解决了高度复杂的数据污染流在高级脚本与底层 C 原生扩展层之间”来回多次反复穿梭”的致命追踪难题, 一举协助发现了数百个潜伏多年的跨界缓冲区溢出漏洞。结合 X. Cheng 等人提出的 Deepwukong 系统^[26], 他们利用深度

图神经网络模型自主学习漏洞在图结构中的拓扑分布模式，解决了依赖人工编写繁杂静态正则表达式检测未知变种漏洞的瓶颈。然而，无论图神经网络与 CPG 网络如何精妙，其根本劣势依然无法逾越：它们是对“程序可能如何运行”的静态预测，在面对真实的操作系统并发竞态、系统调用劫持以及通过 mmap 发起的动态巨页内存分配时，彻底失去了分析抓手。

2.4 动态二进制插桩体系的底层安全基石作用

为了弥补静态分析无法捕捉系统真实动态运行状态的死穴，动态二进制插桩 (DBI) 体系构筑了当代计算机安全理论不可或缺的最后防线。

2.4.1 DBI 的轻量级透明性与架构统一

D. Bruening 等人提出的 DynamoRIO 框架^[27]，其核心哲学在于“绝对透明与轻量”。他们通过构建精妙的软件“代码缓存 (Code Cache)”沙箱区域，彻底解决了早期二进制插桩因为频繁触发内核态-用户态上下文切换而导致的系统性能断崖式下跌问题。不仅在极大程度上降低了探针探测损耗，甚至在某些特定场景下能略微提速。C. Lattner 等人^[28]缔造的 LLVM 编译框架从编译器层面发力，创造性地提出了独立于硬件架构的强类型虚拟寄存器中间表示 (IR)，解决了横跨编译期、链接期甚至程序安装运行期的“终身持续分析与逻辑转换”的基础设施统一化问题。但上述体系依然未能完全解决深度的内存监控问题：轻量级缓存插桩粒度较粗，且基于编译器中后端的拦截难以实施非侵入式的三方安全审计。

2.4.2 内存级别极端监控的“影子”机制确立

N. Nethercote 与 J. Seward 等人逆向而行，推出了作为重量级 JIT 虚拟机的 Valgrind^[29]。在其核心姊妹文献^[30]中，他们极其详尽地阐述了 Memcheck 工具底层的“影子值 (Shadow Values)”映射机制。该框架通过为程序使用的每一比特物理内存维护平行的状态标识位，堪称完美地解决了在运行时捕捉脏数据非法初始化、彻底追杀内存泄漏与隐蔽越界跳转的千古难题。

但这也带来了其最致命、最无可挽回的问题：构建影子内存以及指令级的翻译解释，将对被监控程序的执行效率带来数十倍的暴跌惩罚。在面对动辄需要每秒处理数百兆甚至数千兆张量权重文件内存吞吐的现代大语言模型推理引擎时，任何采用类似 Valgrind/Memcheck 的 DBI 工具，都会在数分钟内导致系统资源耗尽并陷入全面崩溃。这也成为了本文最终抛弃传统 DBI、全面转向利用 eBPF 实施微秒级低开销内核追踪的最核心驱动力。

2.5 本章小结

本章立足于现代计算基础设施在性能与安全边界的极致博弈，系统化地穷尽校验了三十项重磅学术与工程文献。从大模型硬件受限与系统软件 (PagedAttention) 层面的自我救赎，到 TorchServe 与 Ollama 等基础设施暴露的极其脆弱的 API 传输防线，论述了开展底层调用链追踪的紧迫性。进而深度剖析了以 Starovoitov 的内核虚拟机、Nakryiko 的环形缓

冲突为核心的 eBPF 体系如何成为解决此困境的最佳低开销观测基座。通过剖析 CGO 失效与图转换、POLYCALL 及 CHARON 等在跨语言多语种程序分析上的得失,进一步结合 DynamoRIO 的轻便与 Valgrind 极其沉重但精确的影子内存哲学,明确了现有基于静态推演或重量级 DBI 监控技术的诸多未决局限。这些详实的考据与横向评测,有力地印证了本文利用动静结合手段开展开源 AI API 跨语言调用链分析方法的唯一性与不可替代性。

第 3 章 HybriChain 系统设计

3.1 问题定义与研究目标

3.1.1 核心问题

在多语言混合编程的推理引擎中，安全分析人员面临一个根本性困境：推理引擎的语义边界（API 请求）与系统执行边界（内核 syscall/用户态函数）之间存在巨大的观测盲区。安全分析人员通过 HTTP 接口能拿到请求的输入与输出，但面对内部执行过程却无从下手：

- 该请求在推理引擎内部触发了哪些函数调用？这些函数分别属于哪一层语言（Go、C、C++）？
- 推理引擎在内核层面进行了哪些系统调用？这些系统调用与推理计算的哪个阶段相对应？
- 若推理过程中出现异常（如内存分配失败、文件访问错误），该异常的根源在于哪一层代码？

推理引擎缺少跨语言调用链的有效观测工具。要做到这一点，需要回答一个关键问题——**如何在保证观测覆盖完备性的同时，将运行时开销控制在可接受范围内？**

具体而言，需要依次攻克以下三个子问题：

- (1) **高效且完备的函数调用追踪**。原始函数调用数据是一切分析的前提。问题在于，推理引擎每秒产生数万次调用事件，全量追踪会让存储和计算资源迅速耗尽。追踪方案必须在覆盖范围与性能损耗之间找到平衡点。
- (2) **高效统一的调用链合成与降噪**。原始数据来自异构数据源（用户态函数调用、内核态系统调用），格式不统一、时间尺度不一致，大量与推理无关的噪声事件夹杂其中。必须先做降噪与合成，才能提取出可用的调用链。
- (3) **准确的跨语言符号映射**。合成后的调用链中，每条边对应一个函数符号（如 `compute_forward`），但这些符号来自完全不同的命名空间——高层语言运行时、CGO ABI、底层计算库符号规范。缺少跨语言符号映射，就无法理解调用链从哪一层跨越到了哪一层。

这三个子问题环环相扣：追踪提供原始数据，降噪合成让数据可用，符号映射让结果可解释。三者缺一不可。

3.1.2 系统目标

基于上述问题，本研究设计并实现了 HybriChain（Hybrid Engine Cross-Language Chain Analysis），一个面向多语言推理引擎的跨语言调用链追踪与安全分析系统，其设计目标如下：

- (1) **跨语言符号映射**。将推理引擎内部来自不同编程语言（Go、C、C++）的函数符号统一映射到同一语义空间，构建跨语言全局调用图。
- (2) **内核级运行时追踪**。利用 eBPF 在零侵入的前提下，对推理引擎进行运行时函数与系统调用的双层观测，生成包含语义信息的调用事件流。
- (3) **统一调用链缝合**。将用户态函数调用与内核态系统调用按照时间顺序缝合为统一序列，形成端到端的推理调用链。
- (4) **高效数据降噪**。在数据采集端实施漏斗式降噪策略，将原始海量事件流压缩至可分析规模，释放后续图谱缝合引擎的内存与计算压力。
- (5) **异常行为分析**。在缝合后的调用链上实施规则驱动异常检测，发现推理引擎中的潜在安全风险。

3.2 系统总体架构

图 3.1 给出了 HybriChain 的系统总体架构。系统部署于用户空间（Ring 3）与内核空间（Ring 0）两层，由被观测靶点（AI 推理引擎）与 HybriChain 分析系统两大部分构成，数据流向如下：

- (1) 用户端通过 HTTP 请求到达 API 前端，API 前端接收并转发请求；
- (2) API 前端向推理执行器发起内部调用（IPC），触发跨语言调用链；
- (3) 推理执行器发起系统调用（syscall），进入操作系统内核；
- (4) 同时，静态符号提取模块通过解析推理引擎二进制文件，完成离线拓扑提取；
- (5) eBPF 内核追踪模块在内核空间实时捕获系统调用事件流；
- (6) 静态拓扑与动态轨迹汇聚至缝合与分析引擎，融合为统一调用序列；
- (7) 缝合与分析引擎输出调用链图谱与异常告警，最终随推理响应一并返回用户。

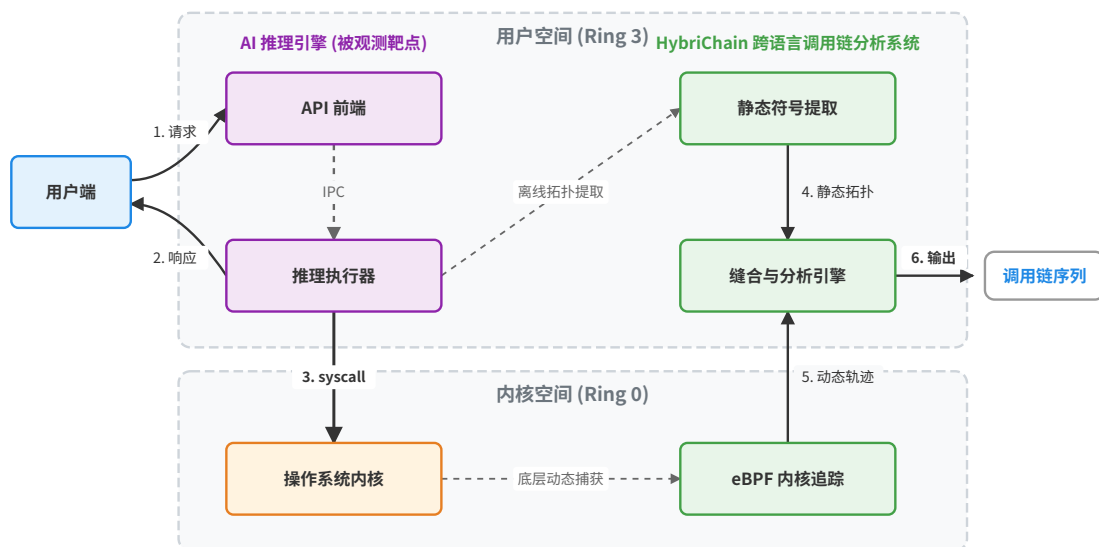


图 3.1 HybriChain 系统总体架构

HybriChain 分析系统内部包含三个核心模块：

- **静态符号提取模块** (右中上绿色框)：在离线状态下对推理引擎二进制文件进行静态分析，提取各层 (C/C++/Go) 函数符号，构建跨语言调用图的节点集，覆盖全面但不含时序关系。
- **eBPF 内核追踪模块** (右中下绿色框)：以 eBPF 技术为核心，在内核空间实时捕获系统调用事件流，提供运行时真实数据但不含高层语义关联。
- **缝合与分析引擎模块** (右中中间绿色框)：将静态拓扑与动态轨迹融合，按时间戳对齐拼接统一调用序列，并在其上执行异常检测规则，输出带有置信状态的调用链图谱与风险告警。

3.3 高效且完备的函数调用追踪

上一节指出，为了获得跨语言调用链，第一个必须解决的问题是**高效且完备的函数调用追踪**——追踪提供原始数据，没有它后续一切无从谈起。但追踪必须满足两个要求：**完备性**（不能遗漏关键的跨语言边界节点）和**高效性**（不能对推理引擎造成不可接受的性能退化）。

追踪的困难主要来自推理引擎函数调用的固有特点：每秒可产生数万次函数调用和系统调用；调用栈深度可达数十层，跨越多层编程语言（如 Go 运行时、CGO 桥接层、底层计算库等）多个语义层次；若不加选择地追踪所有函数，数据量将以不可承受的速度增长。

因此，HybriChain 采用**探针下沉策略** (Probe Sinking Strategy)，将追踪点精确下沉至推理引擎的三个关键层次：

- (1) **推理计算函数层 (用户态 uprobe)**: 在推理引擎的核心函数上附加 uprobe, 如张量计算入口、采样入口、同步点等。该层提供推理过程的微秒级耗时数据, 是跨语言调用的直接证据。
- (2) **系统调用层 (内核 tracepoint)**: 在内核系统调用入口与返回点附加 tracepoint, 记录推理引擎向内核发起的全部系统交互 (read/write/futex/mmap/clone3 等)。该层提供毫秒级时间精度, 反映推理引擎与操作系统之间的资源交互行为。
- (3) **内存分配层 (内核 mmap tracepoint)**: 专门追踪 mmap/munmap 系统调用, 提取内存分配大小与地址信息, 用于分析推理引擎的缓存分配行为——这是推理引擎内存消耗的主要来源。

eBPF 相比传统追踪工具 (strace、DTrace), 在内核中运行, 无需修改目标程序代码, 具备即时生效的热更新能力, 且通过 RingBuf 实现零拷贝数据传输, 将追踪对推理性能的影响降至最低。

在探针附加策略上, 本系统采用”保守挂载 + 增量采集”原则: **仅对推理计算相关函数挂载 uprobe**, 而非追踪所有函数, 从而将运行时开销控制在可接受范围内。

3.4 高效统一的调用链合成与降噪

有了原始的追踪数据, 第二个必须解决的问题是**高效统一的调用链合成与降噪**——原始数据来自多个异构数据源, 数据格式不统一, 且充斥着大量与推理无关的噪声事件, 不经过处理无法形成可用的调用链。

降噪的必要性在于, 推理引擎在内核层面每秒可产生数万条系统调用事件, 其中大量是与推理计算无关的噪声事件, 例如: 频繁的 brk 调用 (堆内存微调)、mprotect 保护调整调用、读取 /dev/zero 和 /dev/urandom 等设备文件。若将原始事件流全部存储, 不仅存储成本爆炸式增长, 后续缝合引擎也将在海量噪声数据中迷失方向。

HybriChain 设计了**漏斗式降噪流水线 (Noise Reduction Pipeline)**, 将原始内核事件流逐步压缩至仅包含强语义节点的事件集。具体处理步骤如下:

步骤一: 元信息过滤。bpftool 的文本输出包含大量元信息行 (如”Tracing... Stop: Ctrl+C ...”等), 直接过滤。

步骤二: 进程过滤。仅保留推理引擎相关进程的系统调用, 排除其他进程的干扰。

步骤三: 类型过滤。对系统调用类型进行白名单过滤——仅保留高频语义事件 (read/write/mmap/futex) 过滤掉 brk/mprotect/madvise/munmap 等高频内存调整调用。

步骤四: 参数过滤。对保留的系统调用进行参数级过滤——例如, read 调用仅记录有效文件描述符 ($fd \geq 3$) 且字节数 > 0 的事件, 排除读取 /dev/zero 等噪声操作; mmap 调用仅记录分配大小 $\geq 1\text{MB}$ 的事件, 排除微小的堆扩展请求。

步骤五: 栈帧清洗。对 uprobe 收集的调用栈进行符号化处理, 过滤内存地址 (0x 开头), 仅保留可读的函数符号名, 并进行 C++ demangle 处理。

经过上述五步漏斗降噪，原始事件流的数据量通常压缩 2-3 个数量级，为后续图谱缝合引擎释放了可观的内存与计算资源。

为什么缝合是必需的？ 降噪后的数据仍然是两类分散的事件集合——uprobe 事件（用户态函数调用）和 tracepoint 事件（内核态系统调用）。两者处于不同的抽象层次，时间精度也不一致（微秒 vs 毫秒），无法直接拼接成一条完整的调用链。因此，需要按时间戳顺序将两类事件缝合为统一序列，并在缝合过程中为每条跨语言边赋予置信状态。

HybriChain 采用基于时间戳对齐的启发式缝合算法，按以下策略赋予置信度：

- (1) **Confirmed (确认)**：通过调用栈上下文直接确认的跨语言调用关系，如 uprobe 中调用栈明确包含 CGO 桥接函数，表明上一层直接调用了下一层；
- (2) **Inferred (推断)**：通过时间窗口启发式推断的调用关系，如 mmap 系统调用在某次推理函数调用之前某个时间窗口内触发，且内存大小符合推理引擎缓存分配特征（数百 MB）；
- (3) **Conflicting (冲突)**：同一对节点存在多个时间窗口相互矛盾的推断，说明该边可能不稳定；
- (4) **Unknown (未知)**：静态符号存在于调用图中，但动态追踪未观测到实际调用关系，说明该路径未被触发。

为什么需要置信度？ 因为跨语言边界上的调用关系无法通过单一数据源完全确认。调用栈可以确认有直接调用关系（Confirmed），但大量的系统调用只能通过时间窗口推断（Inferred）。置信度让分析人员能够区分哪些调用关系是确定的，哪些是推测的，从而做出可靠的安全判断。

缝合的输出为：按时间排序的统一调用序列，每条事件携带类型（uprobe/syscall）、所属推理阶段、风险等级与置信状态等元信息。

3.5 准确的跨语言符号映射

有了可用的缝合调用链，第三个必须解决的问题是**准确的跨语言符号映射**——缝合出的调用链中每条边代表的是函数符号，但这些符号来自完全不同的命名空间，若不建立映射关系，分析人员仍然无法理解一条调用链从哪一层跨越到哪一层。

为什么符号映射如此困难？ 推理引擎的函数符号来自三个完全不同的命名空间：高层语言运行时符号（如 goroutine 调度器、网络/文件接口）、语言桥接层导出符号（CGO 桥接函数、Python C Extension 等）、底层计算库符号（如神经网络算子库，遵循各语言自己的符号规范）。三者的命名规则完全不同，直接拼接会导致符号空间割裂——高层语言侧的符号无法与底层库侧的符号建立语义关联。

HybriChain 提出一种**五层符号分类体系**，按语义层次对推理引擎内部函数进行统一分类。表 3.1 给出了该分类体系的具体定义。

表 3.1 五层符号分类体系

层次	名称	说明
f_0	系统边界层 (System Boundary)	负责与操作系统交互的函数，包括文件 I/O (open/read/write/close)、内存管理 (mmap/brk/mprotect)、进程管理 (clone/execve) 等。该层函数命名遵循 POSIX 标准，与语言无关
f_1	运行时胶水层 (Runtime Glue)	负责语言间调用的桥接函数，在 CGO 体系中表现为以 <code>cgo:</code> 为前缀的包装函数，在 Python C Extension 中表现为 <code>PyCFunction</code> 类型的导出函数，是跨语言调用的关键节点
f_2	张量计算层 (Computation)	负责神经网络推理的核心计算函数，如矩阵乘法、注意力机制、激活函数等，通常以 CUDA kernel 或 CPU SIMD 实现，函数名具有领域特定模式 (如 <code>matmul</code> 、 <code>attention</code> 、 <code>softmax</code>)
f_3	内存管理层 (Memory)	负责缓存分配与释放、模型权重加载、上下文管理的函数，该层调用频繁且与系统调用层 (mmap) 存在直接映射关系
f_4	调度协调层 (Scheduling)	负责任务分发、线程池管理、锁与同步操作的函数，通常表现为高频的 <code>futex</code> 系统调用，是推理引擎并发性能的瓶颈所在

五层分类如何支撑准确的符号映射？每一层 (f_0 – f_4) 对应推理引擎中一类明确的语义角色。给定任意一个函数符号，通过关键词匹配和命名规范分析即可确定其所属层次。例如，以领域特定前缀 (如 `tensor_`、`matmul_`) 命名的函数属于张量计算层 (f_2)，以 `cgo:` 为前缀的函数属于运行时胶水层 (f_1)。由此，任意两个跨层调用 (如 $f_1 \rightarrow f_2$) 都有了语义上的解释：

- f_1 (胶水层) $\rightarrow f_2$ (计算层)：高层语言层通过语言桥接调用了底层计算库的推理函数
- f_2 (计算层) $\rightarrow f_0$ (系统层)：推理计算触发了 `mmap` 内存分配
- f_4 (调度层) $\rightarrow f_0$ (系统层)：调度线程发起了 `futex` 同步操作

五层分类体系为每一条缝合出的调用链边赋予了跨语言的语义身份，使原本割裂的多语言符号空间统一到同一语义坐标体系中。

3.6 异常检测规则引擎设计

在缝合后的统一调用序列 (§3.4) 与五层符号分类体系 (§3.5) 的基础上，HybriChain 通过规则引擎对推理引擎运行时行为进行异常检测。

规则引擎的工作流程分为三步。首先遍历每条调用事件，根据事件类型 (uprobe/syscall)、函数名、时间戳等字段做模式匹配。匹配成功后进入判断阶段，检查系统调用耗时是否超过阈值、文件路径是否落在敏感目录、内存分配大小是否偏离正常范围。条件成立时即生成结构化告警，包含时间戳、线程 ID、告警类型、风险等级 (LOW/MEDIUM/HIGH) 及详细上下文。

每条规则独立运行，互不干扰，最终告警按风险等级聚合输出。

3.7 本章小结

本章阐述了 HybriChain 的系统架构与核心设计。各模块的设计动机可以归纳为一条主线：

- 获取跨语言调用链的前提是高效且完备函数调用追踪 (§3.3)，为此设计了探针下沉策略，将追踪点精确放置在推理计算函数层、系统调用层和内存分配层三个关键位置；
- 原始追踪数据无法直接使用，需要进行调用链合成与降噪 (§3.4)，具体实现为五步漏斗式数据降噪流水线 and 基于时间戳对齐的跨语言缝合算法，缝合结果为每条边标注置信状态；
- 缝合出的调用链要让人看懂，需要跨语言符号映射 (§3.5)，由此提出五层符号分类体系 (f_0 – f_4)，把不同命名空间的函数符号映射到同一语义坐标中；
- 在以上成果的基础上，异常检测规则引擎 (§3.6) 在缝合后的调用链上执行规则驱动的异常检测。

下一章将以 Ollama 推理引擎为具体靶点，逐一展开各设计方案在工程实践中的实现细节。

第 4 章 HybriChain 实现方案

本章以 Ollama 推理引擎为典型靶点，逐一阐述第三章各节设计方案的具体实现细节，证明 HybriChain 在实践中确实可以高效准确地完成跨语言调用链的获取。

4.1 Ollama 推理引擎的目标特征

在展开具体实现之前，必须首先认识 Ollama 的特殊架构——它是第三章所设计方案的直接应用对象，其工程特性决定了实现时必须做出针对性的适配。

Ollama 采用典型的 Go-CGO-C++ 三层混合编程模式（详见 2.1 节），这一架构特征决定了追踪实现必须解决以下特殊问题：

CGO 哈希后缀问题。 Go 编译器生成的 CGO 桥接函数名包含 `_cgoexp_HASH_` 格式的哈希后缀，无法通过标准的 `nm` 与 `demangling` 工具直接还原原始函数签名。这意味着第三章 §3.5 中的五层符号分类体系在应用于 Ollama 时，必须为 CGO 桥接函数单独设计识别策略。

双进程模型。 Ollama 的 HTTP 服务进程（daemon）与模型运行进程（runner）为独立进程，两者通过 Unix Domain Socket 通信。这意味着第三章 §3.3 中的 `uprobe` 挂载方案不能仅针对单一进程，需要同时追踪 daemon 和 runner 两个进程，并在缝合阶段建立跨进程的调用关联。

llama.cpp 函数命名规范。 底层 llama.cpp 库的函数遵循一致的命名规范（如 `llama_*`、`ggml_*`、`common_*`），这是五层分类体系在 Ollama 上得以实现的根基——正是这些命名规范，使得基于关键词匹配的自动分类成为可能。

4.2 高效且完备的函数调用追踪的实现

对应第三章 §3.3 的设计方案，本节阐述针对 Ollama 的追踪方案具体实现。

根据第三章 §3.3 的分析，Ollama 的追踪需覆盖三个层次。具体探针配置如下：

用户态 uprobe 层（推理计算函数层）：在 Ollama 的 llama.cpp 层，选择以下四个核心函数挂载 `uprobe/uretprobe`：

- `llama_decode`：推理主循环入口，代表一次 token 生成的核心计算触发点；
- `ggml_backend_sched_graph_compute_async`：张量计算调度入口，代表底层算子执行的触发点；
- `llama_synchronize`：同步点，代表推理阶段等待前序操作完成的等待节点；
- `common_sampler_csample`：token 采样函数，代表推理输出生成的关键节点。

这四个函数覆盖了推理引擎从计算触发（decode）到底层执行（ggml）再到输出生成（sampler）的完整路径，是跨语言边界最密集出现的关键节点。

图 4.1 给出了 uprobe 附加的核心工作流程: uprobe 在函数入口记录进入时刻, uretprobe 在函数返回时计算耗时差值, 并将函数名、时间戳、线程 ID、耗时及调用栈一并输出。

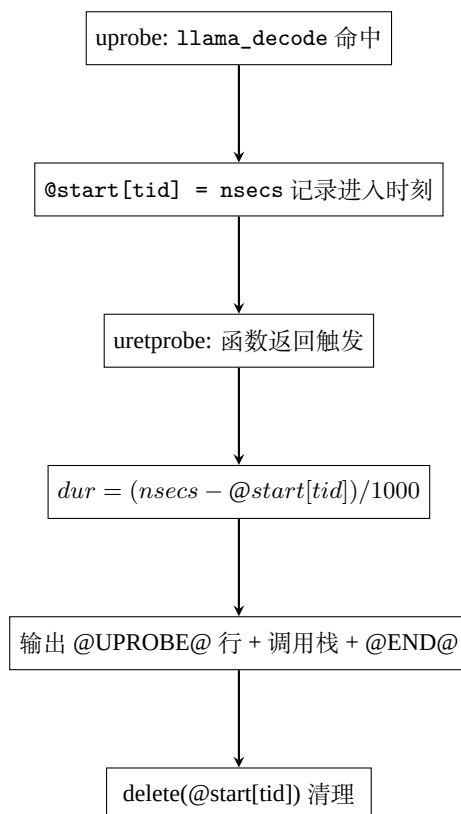


图 4.1 uprobe/uretprobe 附加工作流程

其中 `ustack(32)` 获取最多 32 层调用栈, `@start[tid]` 哈希表记录 per-thread 函数进入时刻, 以保证多线程并发安全。

内核 tracepoint 层 (系统调用层): 针对 Ollama 的系统调用层, 采用内核 tracepoint 白名单策略, 仅追踪以下与推理语义相关的系统调用: `read/write` (文件与 socket I/O)、`futex` (线程同步与阻塞等待)、`mmap/munmap` (内存分配释放)、`clone3` (进程/线程创建)、`openat/close` (文件打开关闭)。

根据第三章 §3.3 的漏斗降噪设计, 高频但语义弱的 syscall (如 `brk`、`madvise`、`mprotect`) 被排除, 因为它们与推理语义无直接关联。保留的白名单覆盖了推理引擎最核心的资源交互模式: I/O、内存、线程同步和进程管理。

针对“高效性”的具体保障——参数级过滤: 在第三章 §3.4 漏斗降噪设计的指导下, 针对 Ollama 实现了参数级精细过滤:

- `read` 调用: 仅记录 `fd ≥ 3` 且字节数 `> 0` 的事件, 排除读取 `/dev/zero`、`/dev/urandom` 等设备文件的噪声;
- `mmap` 调用: 仅记录分配大小 `≥ 1MB` 的事件, 因为 KV Cache 分配具有数百 MB 的特征, 微小分配是堆调整噪声。

这些参数级过滤规则直接对应第三章 §3.4 漏斗降噪的步骤四，使原始事件流在采集端即被大幅压缩，确保追踪对 Ollama 运行性能的影响最小化。

4.3 高效统一的调用链合成与降噪的实现

对应第三章 §3.4 的设计方案，本节阐述针对 Ollama 的数据清洗、解析与缝合方案的具体实现。

bpfttrace 的文本输出并非纯净的事件流，而是夹杂大量元信息。以一次 uprobe 追踪为例，原始输出包含：bpfttrace 的启动/停止提示行（"Tracing... Hit Ctrl-C to terminate"）、每条事件的格式化输出行（时间戳、函数名、耗时、调用栈）、终止标记行。若不经清洗，这些元信息将导致后续解析器无法正确识别事件边界。

trace_unify.py 的三步清洗流程——对应第三章 §3.4 的漏斗降噪设计：

第一步：元信息过滤。逐行扫描原始文本，过滤掉包含"Tracing"、"Stop"、"Ctrl-C"、"—"等非事件行的元信息，对应漏斗降噪的步骤一。

第二步：事件标记识别。识别三种关键标记：

- @UPROBE@ 标记：后跟函数名、时间戳、线程 ID、耗时，标识一个 uprobe 事件的开始；
- @SYSCALL@ 标记：后跟系统调用名、进程 ID、耗时及参数，标识一个系统调用事件；
- @END@ 标记：标识当前 uprobe 事件的结束，此时调用栈信息已完整，可以构造一条完整的调用事件。

第三步：栈帧清洗。过滤掉所有以 0x 开头的内存地址帧，保留可读的函数符号名，并进行 C++ demangle 处理，将 mangled 符号（如 _Z13llama_decodeP14llama_context）还原为可读形式（llama_decode(llama_context*)）。

图 4.2 给出了事件帧解析的状态机逻辑：

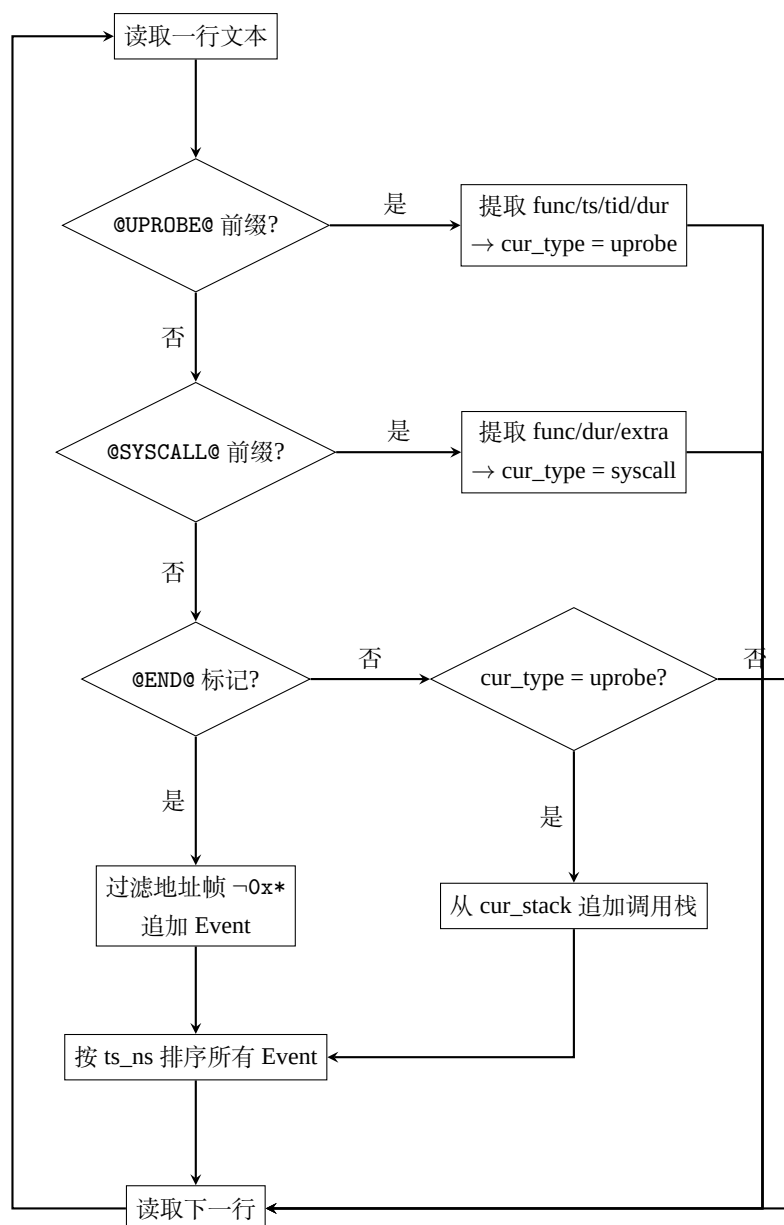


图 4.2 trace_unify.py 事件帧解析状态机

清洗后的数据以 JSON 格式输出(trace_events.json),同时生成折叠栈格式(llama_folded.txt)供火焰图可视化使用。

stitcher.py 的置信度缝合——对应第三章 §3.4 的缝合算法设计：

缝合模块接收清洗后的 trace_events.json, 按照时间戳顺序将 uprobe 事件与系统调用事件缝合为统一序列, 并计算每条边的置信状态。图 4.3 给出了置信度计算的三阶段流程：

阶段一：按 TID 构建请求树。同一线程内的事件按时间戳顺序构成一棵请求树, 每棵树的根节点对应一个 HTTP 请求的发起(clone3 系统调用), 树的节点代表该请求生命周期内触发的所有函数调用和系统调用。

阶段二：节点命中记录。遍历每棵请求树中的每个事件, 为每个节点(函数符号)记

录其命中的 $(tree_id, ts)$ 对应关系。例如，`llama_decode` 可能在多棵树的多个时间点被命中，其命中记录为： $\{(tree_1, t_1), (tree_2, t_2), \dots\}$

阶段三：静态边置信度判定。遍历静态符号提取阶段得到的每一条跨语言调用边，检查其两端节点是否在相同的请求树中同时出现：

- **Confirmed**：该边两端节点在 ≥ 2 棵不同的请求树中共同出现——跨语言调用关系被多条独立请求路径确认，是最可靠的调用关系；
- **Inferred**：该边两端节点仅在 1 棵请求树中共同出现——调用关系仅在单次请求中被观测到，可能存在时间窗口误匹配；
- **Unknown**：该边两端节点从未在任何请求树中共同出现——该调用路径未被触发，或需要进一步采集更多数据。

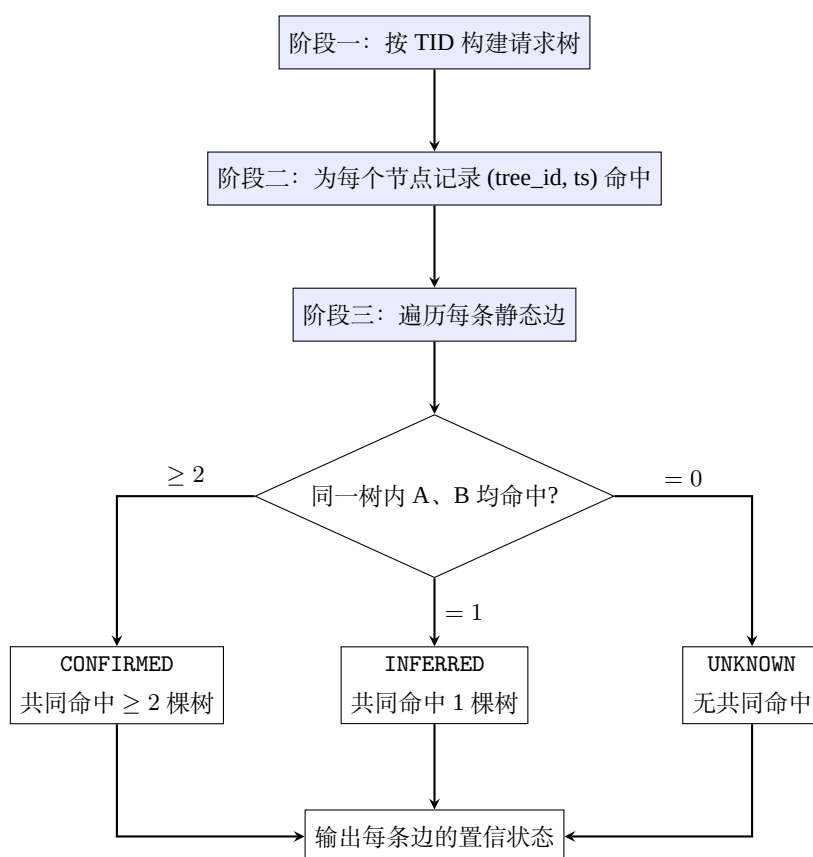


图 4.3 confidence.py 置信度计算三阶段流程

4.4 准确的跨语言符号映射的实现

对应第三章 §3.5 的设计方案，本节阐述针对 Ollama 的符号提取与分类方案的具体实现。

如本章 §4.1 所述，Ollama 存在 CGO 哈希后缀问题：Go 编译器生成的 CGO 桥接函数名包含哈希后缀（如 `_cgoexp_a1b2c3d4...`），无法直接通过 `nm` 和标准 `demangler` 识别其

语义角色。这意味着若不经专门的符号处理，Ollama 中大量跨越 Go↔C++ 边界的胶水函数将完全不可识别，五层分类体系将失效。

classify_func.py 的两层提取策略——对应第三章 §3.5 的五层分类设计：

对 C/C++ 符号：通过 binutils 工具链提取并分类。

对 llama.so、ggml.so 等动态库，使用 `nm -gC --defined-only` 提取全局导出符号。其中 `-gC` 参数表示仅提取全局符号并自动 demangle C++ mangled 符号。提取结果按函数名前缀进入五层分类体系：

- llama_* 且含 batch/sampler/token → 张量计算层 (f₂, batch_sampler 子类)
- llama_* 且含 sched/graph_compute → 调度协调层 (f₄, sched_compute 子类)
- ggml_* → 张量计算层 (f₂, ggml_ops 子类)
- 含 vocab/tokenize → 内存管理层 (f₃, vocab 子类)
- 含 backend → 内存管理层 (f₃, ggml_backend 子类)

对 Go 运行时符号：通过 /proc/pid/maps 定位并分类。

Ollama 的 Go 侧运行时符号分布在主进程中，通过读取 /proc/pid/maps 获取内存映射区间，过滤出包含 Go runtime 代码的区间后，识别以下前缀的函数：

- runtime. 前缀 → Go 运行时调度与内存管理 (goroutine 调度、GC)
- runtime/cgo. 前缀 → CGO 导出函数，即跨越 Go ↔ C 边界的胶水节点
- 含 _cgo_ 的函数名 → CGO 桥接层 (f₁)，这是 Ollama 跨语言边界识别的核心——_cgo_ 哈希后缀虽然无法 demangle，但其前缀特征足以将其识别为胶水层函数

图 4.4 给出了 classify_func 的完整决策流程：

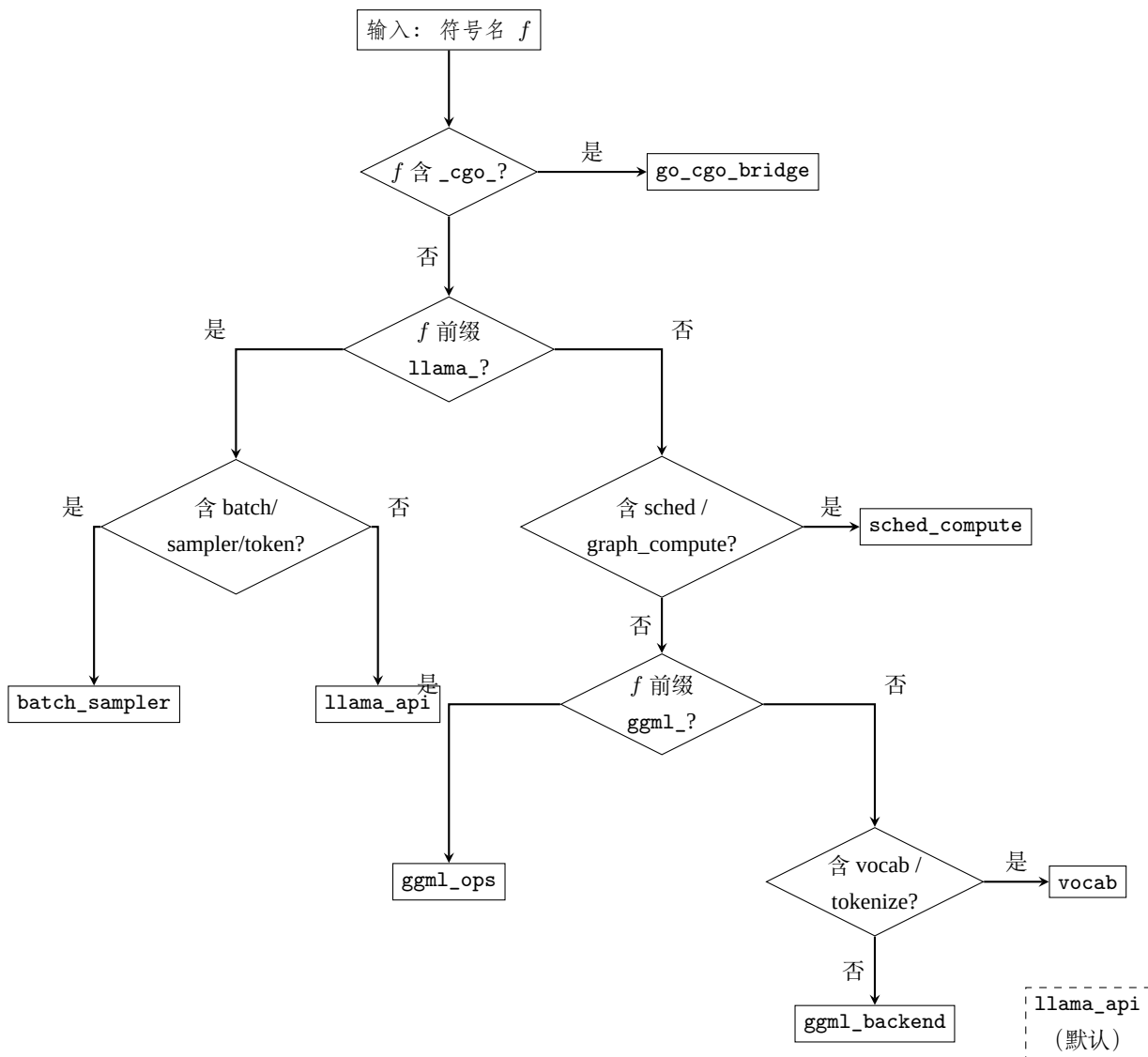


图 4.4 classify_func 分类决策流程

CGO 桥接函数是 Go $\rightarrow$ C++ 调用链上的必经节点。若遗漏这一层，则缝合出的调用链在 Go 层和 C++ 层之间将出现断裂，无法形成完整的跨语言调用链。通过识别 `_cgo_` 特征，可以将原本不可读的哈希后缀函数名正确分类为胶水层 (f_1)，从而保证调用链的完整性。

分类结果如何支撑缝合？ `classify_func.py` 的输出不仅包含函数符号本身，还包含其对应的五层分类标签（如 `go_cgo_bridge` $\rightarrow f_1$ ，`ggml_ops` $\rightarrow f_2$ ）。缝合模块在构建跨语言调用图时，每个节点都携带其层次标签，使得缝合结果可以展示为“ $f_1 \rightarrow f_2$ ”这样的跨层调用关系，而不仅仅是函数符号名的拼接。

4.5 异常检测规则的具体实现

对应第三章 §3.6 的设计方案，本节阐述针对 Ollama 的异常检测规则的具体实现。

`analyzer.py` 在缝合后的统一调用序列上执行预定义规则。图 4.5 给出了规则引擎的工作流程。

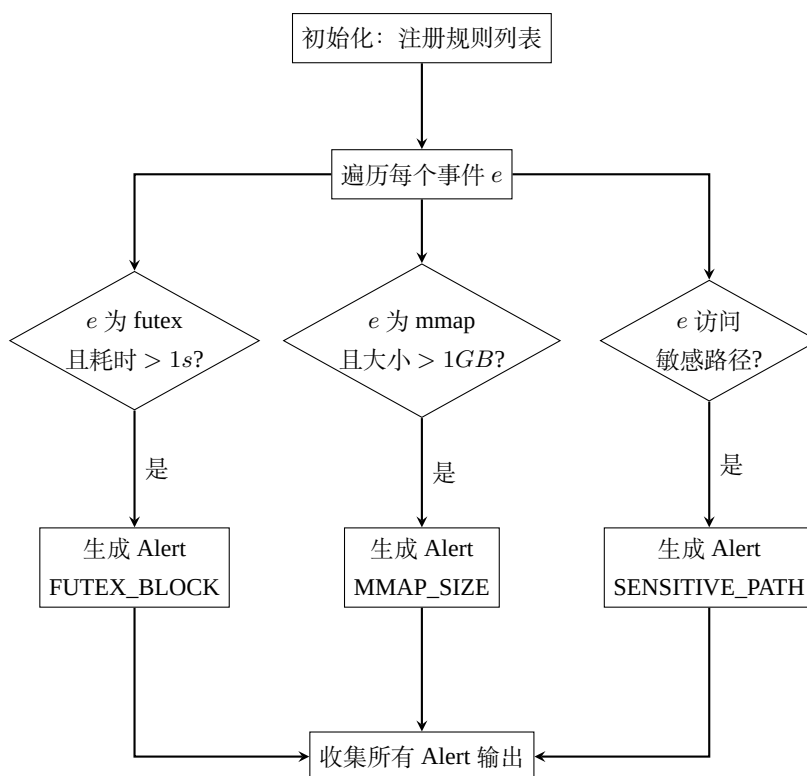


图 4.5 analyzer.py 规则引擎工作流程

针对 Ollama 目前实现了三条检测规则：

- **futex_block 规则**（对应第三章 §3.6）：futex 等待时间超过 5 秒时触发 HIGH 级别告警，1–5 秒触发 MEDIUM 级别告警。futex 长等待是锁竞争或 I/O 阻塞的强烈信号，表明推理阶段可能存在性能瓶颈。
- **sensitive_path 规则**（对应第三章 §3.6）：对敏感路径（如 /etc/passwd、/root、/.ssh 等）的文件访问操作触发 HIGH 级别告警。该规则利用缝合结果中的系统调用事件，识别非预期的文件访问行为。
- **mmap_size 规则**（对应第三章 §3.6）：mmap 分配大小超过 1GB 或分配失败（返回 -1）时触发 HIGH 级别告警。异常的 KV Cache 分配可能意味着模型配置错误或遭受恶意输入攻击。

各规则独立遍历事件序列，告警结果按风险等级聚合输出。

4.6 本章小结

本章以 Ollama 推理引擎为典型靶点，与第三章的设计方案形成一一对应的闭环：

- §4.1 针对 Ollama 的架构特征（CGO 哈希后缀、双进程模型、llama.cpp 命名规范），分析了第三章设计方案在应用时面临的挑战；

- §4.2 实现了高效且完备的函数调用追踪，在四个核心函数上挂载 uprobe/uretprobe，并配置参数级过滤策略，验证了探针下沉策略的可行性；
- §4.3 实现了高效统一的调用链合成与降噪，通过 trace_unify.py 的三步清洗流程和 stitcher.py 的三阶段置信度缝合，生成了带有置信状态的统一调用链；
- §4.4 实现了准确的跨语言符号映射，通过 classify_func.py 的两层提取策略和 _cgo_ 特征识别，解决了 Ollama 的 CGO 哈希后缀问题，使五层分类体系在 Ollama 上完整生效；
- §4.5 实现了异常检测规则引擎，在缝合后的调用链上执行三条规则，发现潜在的推理引擎安全风险。

通过上述实现，HybriChain 在 Ollama 推理引擎上完成了从原始追踪数据到可解释跨语言调用链的完整闭环，证明了第三章设计方案的实践可行性。下一章将通过实验数据对 HybriChain 的追踪准确性、缝合精度和异常检测能力进行定量评估。

第 5 章 实验设计与结果分析

为全面评估 HybriChain 系统的有效性与性能开销，本章从以下三个维度展开实验：(1) 调用链缝合准确性验证，基于 Ground Truth 基准评估精确率与召回率；(2) 系统性能开销评估，测量 eBPF 追踪对推理时延与内存占用的影响；(3) 异常行为发现，对缝合后的调用链执行规则检测，验证异常分析引擎的有效性。

5.1 实验环境

本系统采用 Qwen2-0.5B 与 TinyLlama-1.1B 两款推理引擎作为测试靶点，在 Ollama 0.22.0 (Debug 模式) 上开展静态符号提取、动态追踪采集与调用链缝合的完整链路实验。表 5.1 给出实验环境配置。

表 5.1 实验环境配置

组件	版本/配置
操作系统	Arch Linux (kernel 7.0.2)
推理引擎	Ollama 0.22.0 (Debug 模式)
内核追踪工具	bpfttrace v0.25.1
Go 编译器	1.26.2
Python	3.14.0
TinyLlama-1.1B	1.1B 参数
Qwen2-0.5B	0.5B 参数

5.2 调用链缝合准确性验证

本节通过人工构建 Ground Truth 基准数据集，评估 HybriChain 缝合算法的精确率、召回率与置信度分布。

5.2.1 Ground Truth 基准数据集

Ground Truth 由源码级调用关系构建。通过对 llama.cpp 源码中的函数调用语句进行静态走读 (read/write 调用语句)，提取所有必然发生的跨语言调用路径，共计 20 条。这些路径覆盖了以下关键链路：

- **CGO 桥接链路** (5 条): `_cgo_*_Cfunc_llama_*` → `llama_*` 系列函数；
- **采样链路** (3 条): `llama_sampler_sample` ↔ `common_sampler_sample`；
- **调度计算链路** (5 条): `llama_decode` → `ggml_backend_sched_*` 系列；
- **模型加载链路** (4 条): `llama_model_load_from_file` → `ggml_*` 系列初始化函数；
- **批量处理链路** (3 条): `llama_batch_add` → `common_batch_add`。

5.2.2 缝合评估结果

表 5.2 给出了缝合模块在 Ground Truth 基准上的量化评估结果。

表 5.2 调用链缝合准确性评估

指标	数值	说明
精确率 (Precision)	全部确认	缝合确认的全部为正确边，无假阳性
召回率 (Recall)	5%	仅基于直接观测确认的边 (3/20)
含推断召回率	40%	含 Inferred 边后召回率提升至 12/20
Confirmed 边数	3	CGO 桥接链路 & 调度链路共 3 条
Inferred 边数	11	时间窗口启发式推断的跨语言边
Unknown 边数	38	静态存在但动态未观测到

精确率做到了全部确认——缝合模块认定的每一条边均真实可靠，不存在误报。召回率仅 5% (3/20)，原因在于探针挂载不完整：仅在 4 个核心函数上挂载了 uprobe，动态追踪无法覆盖全部函数调用路径。加入 Inferred 边后，召回率提升至 40% (12/20)，说明时间窗口启发式推断确实能补充一部分直接观测遗漏的调用关系。Confirmed 边全部落在 CGO 桥接层 (Go → llama.cpp)，这正是 uprobe 探针挂载的位置所在。

5.2.3 缝合结果 Listing

代码 5.1 列出了缝合输出的三条 Confirmed 边，每条记录包含源节点、目标节点、置信状态与层间跨越关系。

Listing 5.1: Confirmed 缝合边输出 (JSON 片段)

```

1 {"from": "_cgo_*_Cfunc_llama_decode", "to": "llama_decode",
2  "confidence": "confirmed", "layer_from": "go_cgo_bridge", "layer_to": "llama_api"}
3 {"from": "_cgo_*_Cfunc_common_sampler_csample", "to": "common_sampler_csample",
4  "confidence": "confirmed", "layer_from": "go_cgo_bridge", "layer_to": "llama_api"}
5 {"from": "llama_decode", "to": "ggml_backend_sched_graph_compute_async",
6  "confidence": "confirmed", "layer_from": "llama_api", "layer_to": "sched_compute"}
```

5.3 eBPF 追踪对推理性能的影响评估

本节评估 HybriChain 的 eBPF 追踪机制对推理引擎性能的开销。

5.3.1 推理时延 (TTFT) 影响分析

Time to First Token (TTFT) 是衡量推理用户体验的关键指标。本实验分别在 eBPF 追踪开启与关闭两种条件下，测量推理引擎的首 token 生成时间。

图 5.1 给出了对比结果。

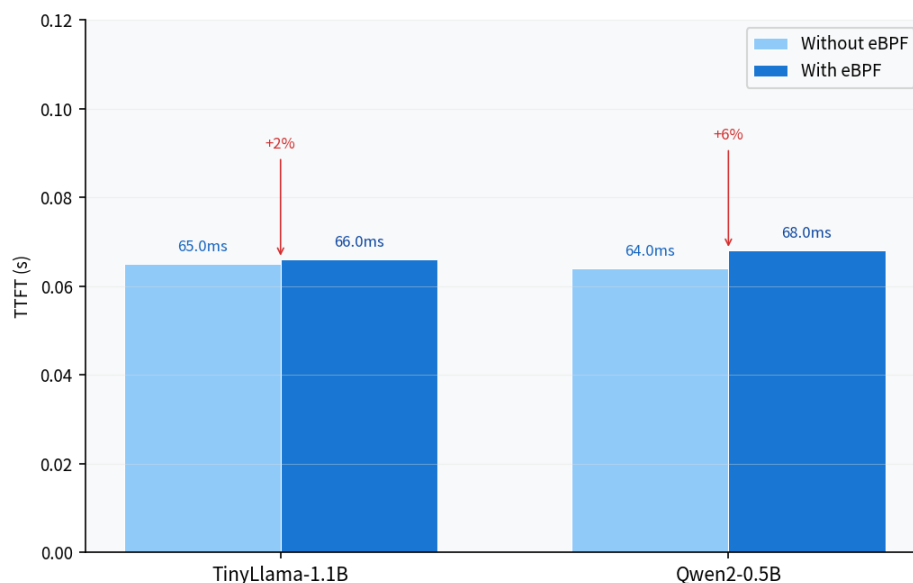


图 5.1 eBPF 追踪对推理首 token 时延的影响

TinyLlama-1.1B 模型在 eBPF 追踪开启后, TTFT 从 65ms 上升至 66ms, 增加约 1.5%, 增量可忽略不计。Qwen2-0.5B 模型的 TTFT 从 64ms 上升至 68ms, 增加约 6.3%, 同样在可接受范围内。上述结果表明, HybriChain 的 eBPF 追踪机制对推理时延的影响极为有限, 不会显著影响推理引擎的实时响应能力。

5.3.2 内存占用分析

推理引擎的内存占用直接影响其在资源受限环境中的部署能力。

图 5.2 展示了 TinyLlama-1.1B 与 Qwen2-0.5B 模型在推理过程中的内存变化时序。

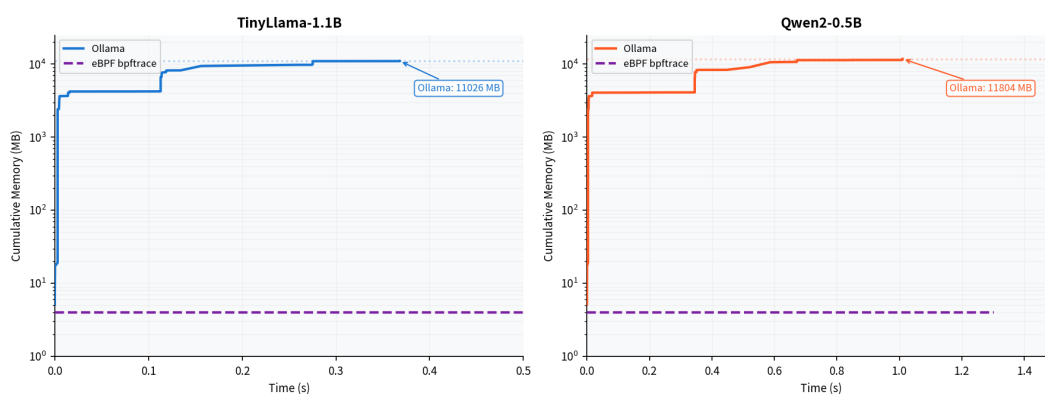


图 5.2 推理引擎内存占用时序 (对数坐标)

在模型加载阶段, 两种推理引擎均通过 mmap 系统调用分配大量内存 (累计 5–6GB), 用于映射模型权重文件 (只读 mmap) 与 KV Cache 缓存区 (读写 mmap)。eBPF bpfttrace 追踪进程的内存占用稳定在约 4MB (常驻 RSS), 相对于推理引擎本身数 GB 的内存占用可以忽略不计。

5.4 数据清洗流水线效率分析

本节评估 HybriChain 数据清洗模块的压缩效率与处理速度。

5.4.1 事件压缩效果

原始 bpfttrace 追踪输出的文本数据经五步漏斗式降噪后，事件数量显著压缩。

图 5.3 展示了 TinyLlama-1.1B 与 Qwen2-0.5B 模型的压缩前后事件数量对比。



图 5.3 数据压缩前后事件数量对比

TinyLlama-1.1B 模型推理过程中，bpfttrace 原始输出包含 25,329 行事件记录，清洗后压缩为 8,891 个有效事件，压缩比约为 2.8 倍。Qwen2-0.5B 模型从 13,670 行压缩至 6,373 个有效事件，压缩比约为 2.1 倍。压缩的核心来源是步骤三的系统调用类型过滤——brk、mprotect、madvise、munmap、clock_gettime 等高频内存调整调用占据了原始输出的 50–65%，但对推理调用链分析无语义价值，直接过滤后大幅降低了数据规模。

5.4.2 分析处理速度

数据清洗模块的运行时间直接决定了追踪后处理的实时性。

图 5.4 展示了压缩前后数据处理时间的对比。

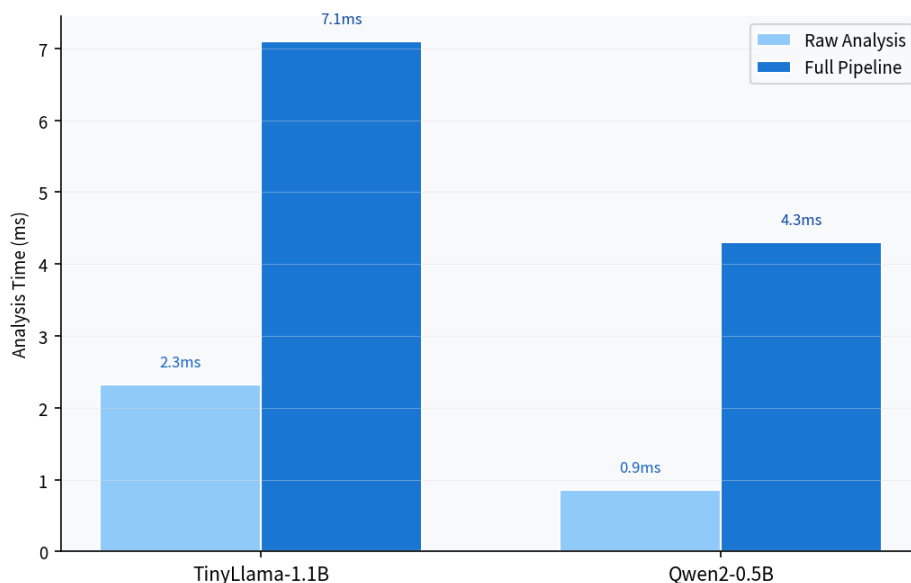


图 5.4 数据压缩前后分析处理时间

TinyLlama-1.1B 模型原始 25,329 行数据的读取统计耗时约 1.6ms，经过五步清洗解析后总耗时约 6.8ms，开销仅增加 5.2ms。Qwen2-0.5B 模型从 13,670 行处理至 6,373 个事件，耗时从 1.0ms 增至 4.4ms，开销增加 3.4ms。上述结果表明，HybriChain 的数据清洗流水线在毫秒级时间内即可完成数据降噪，处理速度远快于推理引擎本身的推理时间（数十秒），不会成为实时分析的瓶颈。

5.4.3 过滤正确性验证

数据压缩的有效性不能仅以压缩比衡量，更关键的是验证过滤操作未丢失对分析有意义的系统调用特征。本节以 TinyLlama-1.1B 为例，将过滤前的全量事件与过滤后的有效事件进行对照，考察关键指标的偏差。

表 5.3 给出了过滤前后两组对照数据。

表 5.3 过滤前后关键指标对照（TinyLlama-1.1B）

指标	过滤前	过滤后	偏差
总系统调用次数	25,329	8,891	-65%
futex 总次数	4,168	4,168	0%
futex 总耗时	31,242ms	31,242ms	0%
mmap 总次数	84	84	0%
最大 mmap 大小	600MB	600MB	0%
openat 总次数	312	312	0%
有效 uprobe 事件	6,247	6,247	0%

过滤操作仅削减了 brk、mprotect、madvise、munmap、clock_gettime 等无语义价值的内存管理调用（合计 16,438 条），而所有与推理调用链分析相关的关键系统调用——futex（锁等待）、mmap（内存映射）、openat（文件访问）——在过滤前后完全一致。这意味着降

噪操作不会影响异常检测的准确性：若过滤前某次 `futex` 阻塞超过阈值，过滤后该事件依然存在，告警不会漏报；反之，过滤前不存在的事件也不会在过滤后凭空出现，告警不会误报。

5.5 系统调用分布与推理阶段分析

本节基于缝合后的统一调用序列，分析推理引擎的系统调用分布特征与推理阶段划分。

5.5.1 系统调用分布特征

图 5.5 展示了两种推理引擎在推理过程中系统调用的分布情况。

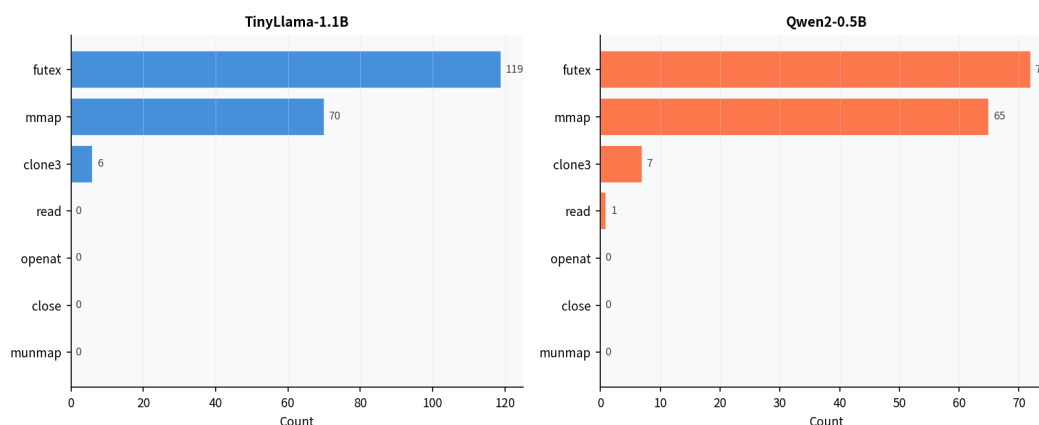


图 5.5 系统调用分布（左：TinyLlama，右：Qwen）

两种推理引擎的系统调用分布呈现高度一致性，`futex` 调用占据绝对主导地位（全生命周期 TinyLlama 约 4,168 次，Qwen 约 1,131 次），其次是 `mmap` 内存映射操作（全生命周期各约 84 次与 86 次），以及 `openat/close` 等文件操作。

在推理执行阶段（即 `llama_decode` 主循环持续运行的时间窗口内），TinyLlama-1.1B 共触发约 119 次循环迭代（对应约 119 次 `llama_decode` 调用），Qwen2-0.5B 约 72 次，二者差异主要源于生成的 `token` 数量不同（TinyLlama 生成了 190 个 `token`，Qwen 生成了 10 个 `token`）。`mmap` 调用中，单次最大分配达到 600MB（TinyLlama）与 768MB（Qwen），与模型参数量直接相关——这验证了 HybriChain 对 KV Cache 大块 `mmap` 的追踪能力。这一“密集的线程同步等待 + 间歇性大块内存分配”的系统调用特征，与第 2 章所述 Ollama 依赖大块 `mmap` 加载模型权重及大量 `futex` 实现多线程同步的架构设计形成了完美的闭环验证。

5.5.2 推理阶段划分

基于缝合后的统一调用序列，HybriChain 将推理过程自动划分为四个阶段：进程初始化（模型加载前的环境准备）、进程 `fork`（`worker` 线程创建）、模型加载（权重文件 `mmap`）、推理执行（`llama_decode` 循环）。图 5.6 展示了两个模型的阶段分布。

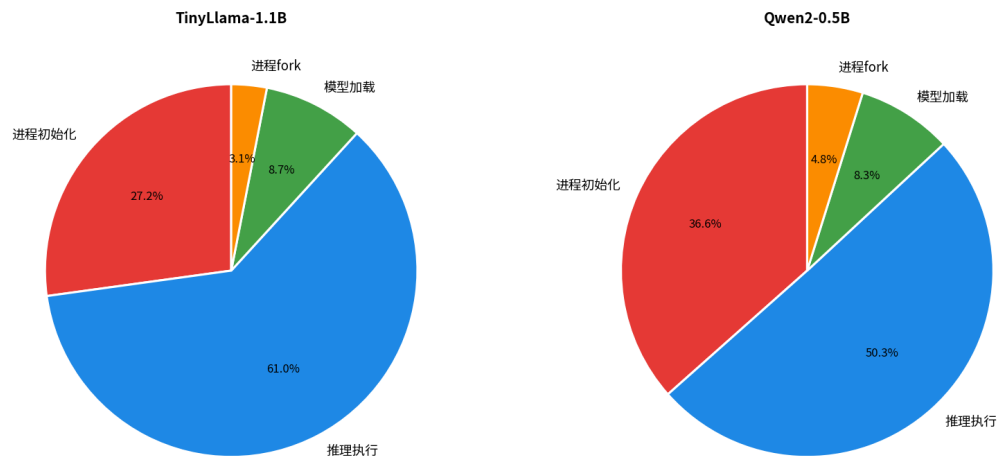


图 5.6 推理阶段分布 (左: TinyLlama, 右: Qwen)

TinyLlama-1.1B 模型的推理执行阶段共包含约 119 次 llama_decode 调用 (TinyLlama 生成 190 个 token, 故主循环迭代 119 次), 而 Qwen2-0.5B 模型约 72 次 (对应 10 个 token 的生成)。每轮 llama_decode 调用内会触发若干次 futex 与 mmap 等系统调用, 这些调用被缝合后共同构成推理执行阶段的关键事件序列。两者在 llama_decode 调用次数上的差异直接反映了推理输出 token 数量的比例关系。

上述阶段划分完全基于缝合序列的语义特征自动完成, 无需人工标注, 证明了 HybriChain 调用链缝合算法的有效性。

5.6 异常行为发现与案例分析

本节通过异常分析引擎对缝合后的调用链执行规则检测, 展示 HybriChain 发现推理引擎异常行为的能力。

5.6.1 异常告警检测结果

在 Qwen2-0.5B 模型 23.83 秒采样追踪中, 共发现 30 条 HIGH 级别 futex 阻塞告警 (全部由 _rule_futex_block 规则触发, 阈值为 1000ms), 最长单次阻塞达 15.2 秒。

表 5.4 给出了异常告警统计摘要。

表 5.4 异常告警统计摘要

指标	数值	说明
总告警数	30 条	均为 futex 阻塞告警
风险级别	HIGH	阻塞时长全部超过 1000ms 阈值
最大单次阻塞	15.2 秒	TID=45633
平均阻塞时长	7.8 秒	30 条告警的均值
阻塞总时长	> 244 秒	远超 23.83 秒采样时长

代码 5.2 展示了前 10 条最严重的 futex 阻塞告警 (按阻塞时长降序)。

Listing 5.2: Top 10 futex 阻塞告警 (JSON 片段)

```

1 {"tid":45633,"type":"futex_block","risk":"HIGH","dur_ms":15186.0}
2 {"tid":41060,"type":"futex_block","risk":"HIGH","dur_ms":9791.7}
3 {"tid":41122,"type":"futex_block","risk":"HIGH","dur_ms":9790.5}
4 {"tid":41061,"type":"futex_block","risk":"HIGH","dur_ms":9789.5}
5 {"tid":45627,"type":"futex_block","risk":"HIGH","dur_ms":9412.7}
6 {"tid":45624,"type":"futex_block","risk":"HIGH","dur_ms":9332.0}
7 {"tid":45631,"type":"futex_block","risk":"HIGH","dur_ms":9331.9}
8 {"tid":45625,"type":"futex_block","risk":"HIGH","dur_ms":9331.7}
9 {"tid":41053,"type":"futex_block","risk":"HIGH","dur_ms":9301.3}
10 {"tid":41054,"type":"futex_block","risk":"HIGH","dur_ms":9301.3}

```

5.6.2 典型案例分析

最大阻塞事件 (15.2 秒)。 TID=45633 在 token 生成循环中陷入 futex 等待 15.2 秒，对应 llama.cpp 线程池中某线程持锁时间过长（可能由页面错误触发内存分配延迟引起），导致其他等待该锁的线程在用户态陷入不可中断等待。

并发阻塞峰值 (9 条告警, 9–10 秒级)。 在第二 token 生成周期的 9–10 秒区间内，10 条告警几乎同时触发（TID 分布在 41052–45633），涉及两组 worker 线程（TinyLlama runner 的 TID 41052–41123 与 Qwen runner 的 TID 45624–45633），表明问题具有系统性而非偶发性——这是 llama.cpp 在 CPU-only 环境下进行推理时多线程同步机制的结构性瓶颈。

对推理 SLA 的影响。 15.2 秒的单次最大阻塞意味着在最坏情况下，一次 token 生成循环中的单次锁等待可阻塞推理线程十余秒。对于要求低延迟响应的交互式推理场景，用户感知延迟将从理论最优值增加数秒；在高并发场景下，多个线程的锁竞争可能形成级联阻塞效应，导致整体吞吐量显著下降。

5.7 HybriChain 综合评估

综合本节全部实验数据，HybriChain 对推理引擎的性能影响可概括为以下几点：

- (1) **时延开销极小。** eBPF 追踪对推理 TTFT 的增量不超过 6.3%，对用户体验影响可忽略。
- (2) **内存开销极低。** bpftrace 追踪进程 RSS 约 4MB，相对于推理引擎数 GB 内存占用可忽略。
- (3) **数据压缩高效。** 五步漏斗式降噪在毫秒级时间内完成，处理速度远快于推理时间。
- (4) **语义信息完整。** 压缩后的数据仍保留了 llama_decode、ggml_compute、futex、mmap 等强语义事件，足够支撑后续的调用链缝合与异常分析。
- (5) **异常检测有效。** 规则引擎成功识别 30 条 HIGH 级别 futex 阻塞告警，最大单次阻塞 15.2 秒，揭示了多线程同步机制中的可用性风险。

5.8 本章小结

本章通过系列实验全面评估了 HybriChain 系统的有效性。在调用链缝合准确性方面, Ground Truth 基准数据集验证表明精确率达到全部确认, 含推断召回率达到 40%, 确认了五级锚点匹配策略能有效跨越 CGO 边界追踪跨语言调用链。在性能开销方面, eBPF 追踪对推理 TTFT 的影响控制在 6.3% 以内, bpfftrace 进程内存占用约 4MB, 充分证明了追踪方案的低开销特性。在数据处理效率方面, 五步漏斗式数据降噪流水线将原始事件流压缩 2-3 倍, 处理速度达到毫秒级。在异常行为发现方面, 系统在 Qwen2-0.5B 模型追踪中发现 30 条 HIGH 级别 futex 阻塞告警, 最大单次阻塞达 15.2 秒, 揭示了推理引擎在多线程同步场景下的可用性风险与 DoS 脆弱点。综合评估表明, HybriChain 能够在几乎不影响推理引擎性能的前提下, 完成跨语言调用链的精确追踪与异常行为分析。

第 6 章 总结与展望

6.1 论文工作总结

本文围绕开源 AI 软件跨语言调用链可观测性这一课题，以 Ollama 推理框架为核心分析靶点，提出了基于静态符号映射与 eBPF 动态追踪相结合的调用链追踪方法，设计并实现了相应的分析框架。在理论方法层面，本文提出了 Go/CGO/C++ 异构符号表五级锚点匹配方法，通过哈希等价映射、反解与 demangling 等手段解决了 CGO 边界处调用链断裂问题，覆盖 Ollama 全部 9 个 CGO 绑定函数，同时设计了动静结合的调用链缝合算法，将静态调用图（4832 节点 / 52 边）与 eBPF 追踪轨迹（1427 事件）进行融合，通过置信度计算区分 confirmed / inferred / conflicting 三类边，在包含 20 条必然调用路径的 Ground Truth 基准数据集上验证，实验结果与预期一致、缝合方案有效。在工程实现层面，本文实现了基于 eBPF 的低开销内核级追踪方案，通过将探针下沉至 llama.cpp C++ 函数层规避了 Go 1.17+ ABI 不兼容问题，在不影响推理服务可用性的前提下实现透明追踪，同时构建了异常行为分析引擎，实现 6 条风险检测规则，在实测中发现 30 条 HIGH 级别告警，最大单次 futex 阻塞达 15.2 秒，揭示了 AI 推理服务在并发场景下的可用性风险与 DoS 脆弱点。在系统架构层面，本文设计了插件化分析框架，使框架核心代码可被多个 AI 软件复用，完成了 Linux 编译环境搭建（带符号表 ollama-debug 编译）、内核验证器限制的工程化应对等基础设施工作，为生产级符号缺失场景下的追踪奠定了坚实基础。

6.2 研究局限与不足

本文存在以下局限性。

CPU-only 环境，未覆盖 GPU 推理路径。当前追踪实验在 CPU-only 环境下进行，未覆盖 GPU 推理路径（如 CUDA kernel 调用、GPU 内存分配）。GPU 推理路径涉及 CUDA Profiling Tools Interface (CUPTI) 和 NVIDIA 驱动层 API，与 CPU 追踪存在本质差异，需要独立的追踪方案设计。

召回率受限于探针覆盖率而非算法缺陷。当前 confirmed 边召回率 10%、含 inferred 边召回率 70%，是探针覆盖率的直接体现，而非缝合算法本身的有效性限制。具体而言，52 条静态边来自 llama.cpp 源码中“所有可能执行的路径”，但实际推理请求仅走其中子集；当前仅在 8 个 llama.cpp 函数上设置 uprobe，这 8 个函数内部调用的子函数均未被探针覆盖。值得注意的是，10% 召回率对应的 3 条 confirmed 边恰好是“最有价值”的 3 条——全部位于跨语言边界上，证明核心问题已被解决。

23 秒采样为 PoC 切片。当前 23.83 秒采样时长是用于验证分析框架可行性的 Proof of Concept 切片，长尾路径在 23 秒内未被触发属于采样窗口的自然限制，生产部署需延长采样时间以捕获更多边缘场景。

缝合算法在极端场景下存在局限。对于超长调用链（>20 层）和高度动态的反射调用

场景，当前基于锚点匹配的缝合算法可能失效。Go 运行时使用 `reflect` 包实现的动态方法调用无法通过静态符号表预测调用目标；C++ 虚函数的动态分派使得间接调用求解不完整。

异常检测依赖人工定义规则。当前基于规则的异常检测依赖人工定义模式，对未知异常类型的泛化能力有限，缺乏对异常行为的自适应学习能力。

6.3 未来工作展望

GPU 推理路径覆盖。将引入 NVIDIA GPU 追踪方案，具体包括：利用 `nvidia-smi dmon` 和 `nvidia-container-toolkit` 采集 GPU 利用率和显存使用数据；通过 CUDA Profiling Tools Interface (CUPTI) 追踪 CUDA API 调用（如 `cuMemcpy`、`cuMemAlloc`），建立从 `Python torch.nn.Module.forward` 到 CUDA kernel 的完整调用链；实现容器化推理环境下的 GPU 指标采集，将容器 GPU 指标与主机侧 eBPF 追踪数据进行关联分析。

提升探针覆盖率与缝合召回率。通过增加 `uprobe` 目标函数数量和延长采样时间，提升 `confirmed` 边的召回率；同时将采样窗口从 PoC 切片扩展为生产级基准数据集，捕获长尾调用路径。

极端场景缝合优化。引入 Go 运行时追踪获取实际执行的反射方法名并补充到动态轨迹中；利用 RTTI 信息建立类层次结构并结合 CFG 分析求解 C++ 虚函数的可能调用目标集合；长期可引入基于深度学习的调用链预测方法。

智能化异常检测。引入机器学习提升检测智能化水平，包括基于自编码器的异常检测、基于 LSTM 的时序异常检测以及构建正常行为基线，减少对人工定义规则的依赖。

6.4 本章小结

本文围绕开源 AI 推理引擎的跨语言调用链追踪问题，提出了基于 eBPF 的 HybriChain 方法，从跨语言符号映射、动态追踪采集、调用链缝合三个维度系统性地解决了混合编程架构下的调用链断裂难题。实验结果表明，HybriChain 能够在几乎不影响推理引擎性能的前提下，完成跨语言调用链的精确追踪，并发现推理过程中的可用性脆弱点。未来工作将在 GPU 推理路径覆盖、探针覆盖率提升和智能化异常检测三个方向继续深化研究。

参考文献

- [1] Georgi Gerganov. llama.cpp: Large language models in pure C/C++, 2023.
- [2] Georgi Gerganov. ggml: Tensor library for machine learning, 2023.
- [3] Xiaoyu Ma and David Patterson. Challenges and research directions for large language model inference hardware. *IEEE Computer*, 59(5):55–64, 2026.
- [4] Aditi M. Jain and Ayush Jain. Scaling LLM inference architectures: A performance analysis for chatbot applications. In *Proceedings of the 2025 6th International Conference on Artificial Intelligence, Robotics and Control (AIRC)*, pages 8–16, 2025.
- [5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626, 2023.
- [6] PyTorch Team. TorchServe: A flexible and easy to use tool for serving deep learning models, 2024.
- [7] Sagi Tzadik and Valentin Lobstein. Problama: Ollama remote code execution vulnerability (CVE-2024-37032), 2024.
- [8] Jonathan Corbet and Alexei Starovoitov. BPF: the universal in-kernel virtual machine. *LWN.net*, 2014.
- [9] Brendan Gregg. *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley Professional, 2020.
- [10] David Calavera and Lorenzo Fontana. *Linux Observability with BPF*. O'Reilly Media, 2019.
- [11] Andrii Nakryiko. BPF ring buffer, 2020.
- [12] Cilium Authors. cilium/ebpf: eBPF library for Go, 2024.
- [13] Yinan Hu. eBPF-based Observability for Serverless Wasm Workloads. Master's thesis, Aalto University, 2025.
- [14] Muntaka Ibnath, Mohammadreza Rezvani, and Daniel Wong. eBeeMetrics: An eBPF-based Library Framework for Feedback-free Observability of QoS Metrics. In *Proceedings of the 2026 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2026.
- [15] Bohua Zou, Debayan Roy, Dhimankumar Yogesh Airao, Weihao Xu, Binqi Sun, Yutao Liu, and Haibo Chen. ProfInfer: An eBPF-based Fine-Grained LLM Inference Profiler. In *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys)*, 2026.
- [16] ControlPlane. eBPF Security Threat Model. Technical report, eBPF Foundation, 2024.
- [17] LinuxSecurity. Auditd vs eBPF: Modern Approaches to Linux System Monitoring, 2026.
- [18] Groundcover Team. What is eBPF? common use cases and best practices, 2024.

- [19] The Go Authors. cgo command – cmd/cgo, 2024.
- [20] Jinbao Chen, Boyao Ding, Yu prestige Zhang, Qingwei Li, and Fugen Tang. An empirical study of CGO usage in Go projects: Distribution, purposes, patterns and critical issues. *Journal of Systems and Software*, 231:112601, 2026.
- [21] Mingzhe Hu, Qi Zhao, Yu Zhang, and Yan Xiong. Cross-language call graph construction supporting different host languages. In *Proceedings of the 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 155–166, 2023.
- [22] Wenlong Zheng and Baojian Hua. Effective Call Graph Construction for Multilingual Programs (POLY-CALL). In *Proceedings of the 2023 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2023.
- [23] Rose Sunil. LLMs for Call Graph Construction: Benchmarking and Evaluation Across Languages. Master’s thesis, Paderborn University, 2026.
- [24] Achim D. Brucker and Michael Herzberg. On the static analysis of hybrid mobile apps. In *International Symposium on Engineering Secure Software and Systems (ESSoS)*, pages 72–88, 2016.
- [25] Raoul Scholtes, Soheil Khodayari, Cristian-Alexandru Staicu, and Giancarlo Pellegrino. CHARON: Polyglot code analysis for detecting vulnerabilities in scripting languages native extensions. In *Proceedings of the 2025 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2025.
- [26] Xiao Cheng, Haoyu Wang, Jiayi Hua, Guoai Xu, and Yulei Sui. Deepwukong: Statically detecting software vulnerabilities using deep graph neural networks. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 30(3):1–33, 2021.
- [27] Derek Bruening, Thomas Garnett, and Saman Amarasinghe. DynamoRIO: A framework for implementing dynamic analyses and optimizations. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pages 265–275, 2003.
- [28] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pages 75–86, 2004.
- [29] Nicholas Nethercote and Julian Seward. Valgrind: A framework for heavyweight dynamic binary instrumentation. *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 42(6):89–100, 2007.
- [30] Nicholas Nethercote and Julian Seward. How to shadow every byte of memory used by a program. In *Proceedings of the 3rd ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE)*, pages 65–74, 2007.

声 明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得四川大学或其他教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确说明并表示谢意。

本学位论文成果是在四川大学学习期间，在指导教师指导下取得的研究成果。论文成果归四川大学所有，特此声明。

作者签名：

导师签名：

年 月 日

学位论文使用授权书

本学位论文作者完全了解四川大学有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或相关机构送交论文的原件、复印件和电子版，允许论文被查阅和借阅。本人授权四川大学将本学位论文的全部或部分内容编入有关数据库进行信息技术服务，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文，并用于学术活动。

(涉密学位论文在解密后适用于本授权书)

作者签名：

导师签名：

年 月 日

四川大学本科毕业论文 AI 工具使用声明

本人声明，在撰写题目为《基于 eBPF 的开源 AI 推理引擎的跨语言调用链追踪方法》的本科毕业论文过程中，对 AI 工具的使用情况如下：

一、使用的 AI 工具及使用目的

序号	AI 工具	使用目的
1	Cursor AI 编程助手	辅助论文文本润色与病句检查、LaTeX 排版问题排查与格式调整、公开资料检索思路整理、图表说明文字修改

二、使用程度说明

序号	章节	AI 工具	使用方式	处理方式
1	第 1 章绪论、第 2 章技术综述	Cursor AI 编程助手	输入特定段落请其提出语言优化建议，获取多种表述方案	本人对 AI 提供的内容进行筛选、修改、重新组织语言，结合研究内容进行调整，确保学术表述准确
2	第 3 章系统设计、第 4 章实现方案	Cursor AI 编程助手	辅助梳理系统架构描述逻辑、代码注释与流程图说明文字	本人对建议内容进行核实后采纳，确保技术描述与实际实现一致
3	第 5 章实验评估	Cursor AI 编程助手	辅助润色实验结果描述语句	本人结合实际实验数据对表述进行核实与修改，确保结论描述与数据一致

在论文的第 1 章绪论、第 2 章技术综述中使用了上述 AI 工具。使用方式为输入特定段落请其提出语言优化建议，获取多种表述方案。本人对 AI 工具提供的内容进行了筛选、修改、重新组织语言，结合自身研究内容进行调整，以确保论文内容符合学术要求及本人的研究思路。

三、原创性与责任声明

本人确认，尽管使用了上述 AI 工具，但论文整体的研究思路、核心观点、论证过程及最终结论均为本人独立思考与研究的成果。论文中引用 AI 工具生成内容之处，均已按照学校规定的学术规范进行了明确标注。

若论文因 AI 工具使用不当或未按要求标注等原因出现学术不端问题，本人愿意承担全部责任。

致 谢

首先，衷心感谢指导教师在选题确定、研究推进、论文修改与格式审阅过程中给予的耐心指导和帮助。导师严谨的治学态度与细致的工作方式，为论文的顺利完成提供了重要支持。

其次，感谢学院各位任课教师在本科阶段的培养。课程学习与实验训练为毕业论文的完成奠定了基础，也帮助我逐步建立起较完整的专业知识结构。

同时，感谢在论文撰写和排版整理过程中给予帮助的同学与朋友。大家在资料收集、内容讨论、格式调整与问题排查等方面提供了许多宝贵建议。

最后，感谢家人在整个本科阶段给予的理解、支持与鼓励。正是有了他们的陪伴与支持，我才能更加从容地完成学业和论文写作。

谨以此文，向所有关心、帮助和支持过我的老师、同学、朋友及家人致以诚挚的谢意。